

Conflict-Free Coloring of Graphs

An expanded walkthrough for undergraduates

Shrey Shah

2026-09-03

Table of contents

1	Why this paper	2
1.1	What this document adds	3
2	What you need to know first	3
3	Intro to Conflict-Free Coloring	7
3.1	A single vertex: $\chi_{CF} = 1$	7
3.2	Complete graphs: $\chi_{CF}(K_n) = 1$	9
3.3	$K_{2,3}$: the first jump to two colors	10
4	Proof 1 - Necessary Coloring	11
4.1	Lemma 3.2 - The Forcing Gadget	11
4.2	Lemma 3.3 - The G_K Hierarchy	12
4.2.1	Why we need this	12
4.2.2	Base cases	13
4.2.3	The recursive step	13
4.2.4	Lower bound	14
4.2.5	Upper bound	14
4.2.6	Counting the vertices	16
4.2.7	Algorithmic bounds	16
4.2.8	Normalizing away the factorials	17
4.2.9	The Pell recurrence	18
4.2.10	The homogeneous part	18
4.2.11	The inhomogeneous part	19
4.2.12	The closed form	19
4.2.13	Growth rate	20
4.2.14	Comparison with the paper	20
4.3	Lemma 3.4 — The Reduction	21
4.3.1	The construction	21
4.3.2	Direction 1: From χ_P of G to χ_{CF} of G'	21
4.3.3	Direction 2: From a χ_{CF} of G' to a χ_P of G	22
4.3.4	Verifying the Reduction is in P	23

4.3.5	Conclusion	24
4.4	Theorem 3.5 — A Sufficient Criterion	24
4.4.1	Notation for this section	24
4.4.2	The proposition	25
4.4.3	Base case: $k = 1$	25
4.4.4	Step 1: G has no K_3 minor, so it has no cycles	25
4.4.5	Step 2: G has no $K_{1,3}$ minor, so $\Delta(G) \leq 2$	27
4.4.6	Step 3: a forest with $\Delta \leq 2$ is a disjoint union of paths	27
4.4.7	Step 4: Every path is conflict-free colorable with one color	28
4.4.8	Distance-3 sets	29
4.4.9	The algorithm	30
4.4.10	Inductive step: $\mathcal{P}(k-1) \Rightarrow \mathcal{P}(k)$	31
4.4.11	Claim A: no set of unseen vertices is a cutset of G	32
4.4.12	Claim B: W has no K_{k+1} minor and no K_{k+2}^{-3} minor	33
4.4.13	Assembling the induction	34
4.5	Corollary 3.6 — The Conflict-Free Hadwiger Theorem	35
5	Hardness	36
6	Outerplanar graphs	36
7	Loose ends	36
8	References	36

1 Why this paper

The quick version: Conflict-Free Coloring describes a situation with interconnected points or nodes, in a graph. The goal is to take each node, and the nodes attached to it and search for a color that only occurs once. In other words, we’re searching for a unique color in that node’s **neighborhood**. Not every node has to be colored, but every node’s neighborhood must contain at least one unique color.

There are numerous reasons we’d want to establish whether a conflict free-coloring was possible, and more frequently how to achieve it with as few colors as possible. The “colors” being referred to are identifiers, and can represent anything from a unique frequency to a data signature like a clearance level. Therefore, finding out how to achieve conflict free coloring with the minimum number of colors, can prove useful in a slew of optimization problems.

A key example would be in the case of smart phones and cell towers. Cities and public planners want to establish as few cell towers as possible (because of how notoriously costly they are), while ensuring that there is no location, lacking a signal. Cell towers have a limited number of frequencies they can use establish network access with your phone. If no cell tower in range uses a unique frequency, then the cell phone will receive multiple frequencies of the same kind at the same time, and these signals will collide, causing interference, and preventing network connection. This use of frequencies extends to almost every form of wireless communication, including in drone swarms,

where robots have to communicate their positions with at least one other nearby drone to prevent collision.

The paper, “Conflict-Free Coloring of Graphs”, by Abel et al. establishes that if a given graph does not contain a complete minor of size $k + 1$ (where a minor is a graph obtained from the initial graph by contracting or deleting an edge or edges, and/or deleting isolated vertices), then the graph is conflict free colorable with k colors for closed neighborhoods. The paper further proves that all planar graphs (which are graphs that can be arranged such that no two edges overlap), are always conflict free-colorable with 3 colors, though less than 3 colors may suffice. The paper also establishes computational complexity for various methods of conflict free coloring with specific constraints. The paper tackles the algorithmic complexity of minimizing the number of colors and the number of colored vertices, for planar graphs (and variants of planar graphs). They finally establish that for open neighborhoods, every planar **bipartite** graph can be conflict free colored with 4 or fewer colors, and every planar graph has a conflict-free coloring with at most 8 colors.

1.1 What this document adds

If any of the above purposes of the paper seemed esoteric or complex, that’s completely normal. The purpose of this document is to simplify this research while exploring it further. This document aims to reconstruct the findings of the paper by more extensively defining key terms, adding more visualizations (especially because they can help abstract thinking about graphs), expanding upon the lemmas, subproofs, and algorithms found throughout the paper.

2 What you need to know first

You will need a working knowledge of very elementary graph theory to understand the concepts. If you need a refresher, this [blog](#) has a quick and holistic one.

Open and Closed Neighborhoods: An open neighborhood of a vertex v is the set of vertices adjacent to v . A closed neighborhood of a vertex v is the set of vertices adjacent to v , and the vertex v itself. Both kinds of neighborhoods are used throughout the paper, and the paper generally splits its various proofs by which type of neighborhood is being used.

Complete Bipartite Graphs: A complete bipartite graph is a graph whose vertices can be split into two disjoint sets, such that for every vertex in one set, there is an edge to every vertex in the other set, and no vertices in the same set are connected by an edge. A complete bipartite graph with m vertices in one set and n vertices in the other is denoted $K_{m,n}$.

Edge Subdivision: For an edge $e \in E(G)$ for a graph G , subdividing $e = uv$ means deleting e ($E(G) \setminus \{e\}$), adding a new vertex w , and adding the edges uw and wv . A graph obtained from G by subdividing edges is called a **subdivision of G**

Edge Contraction: For an edge $e \in E(G)$, where G is a graph and $e = uv$, and for a vertex $w \notin V(G)$, the **edge contraction** of e is the graph obtained by deleting e and deleting the vertices u and v , replacing them with w , and making w adjacent to every neighbor of u and every neighbor of v .

Identifying u and v produces a multigraph, with a loop where e used to be and a parallel pair for each common neighbor; deleting those recovers a simple graph.

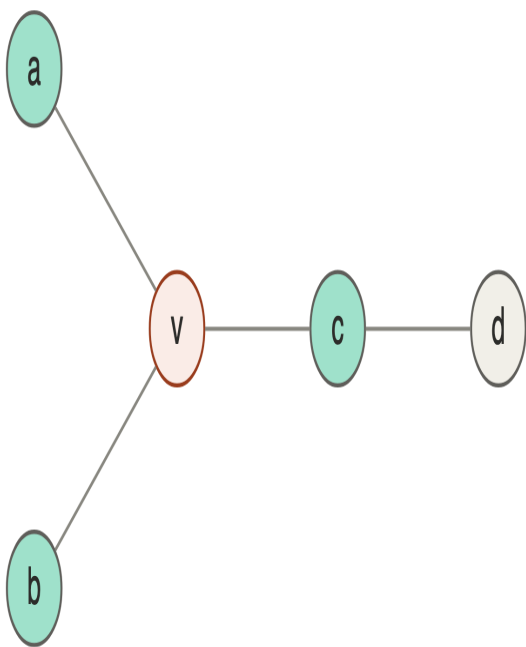


Figure 1: $N(v) = \{a, b, c\}$

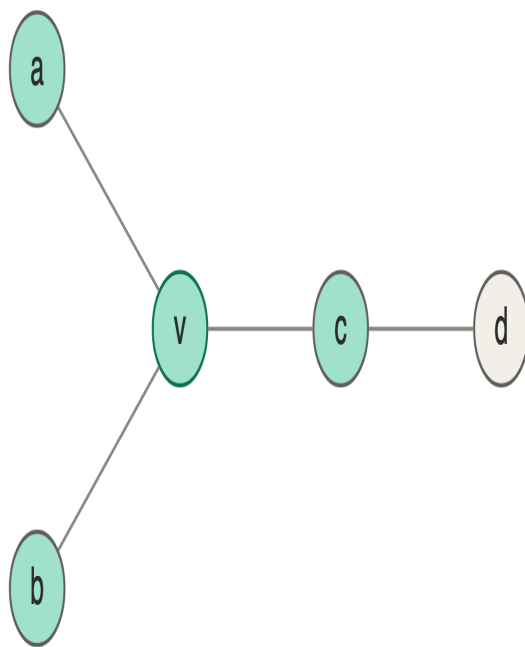


Figure 2: $N[v] = \{v, a, b, c\}$

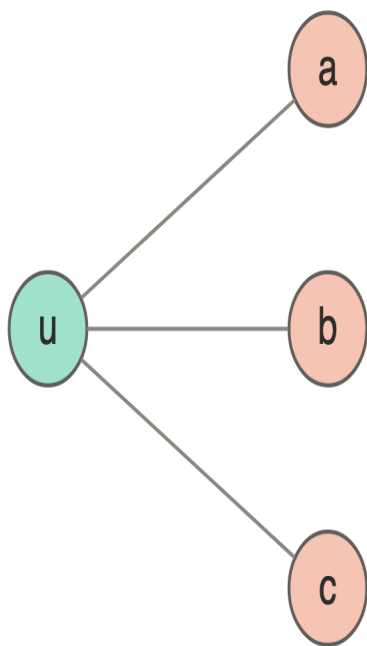


Figure 3: $K_{1,3}$, the star on four vertices

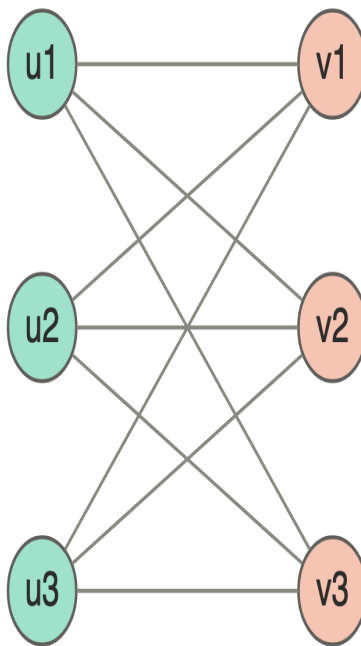


Figure 4: $K_{3,3}$, with all nine edges present

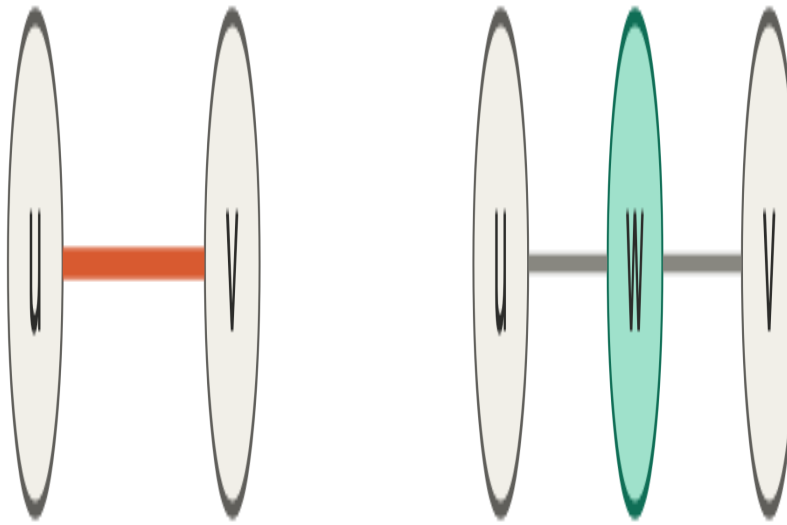


Figure 5: Subdividing $e = uv$ inserts w along the edge

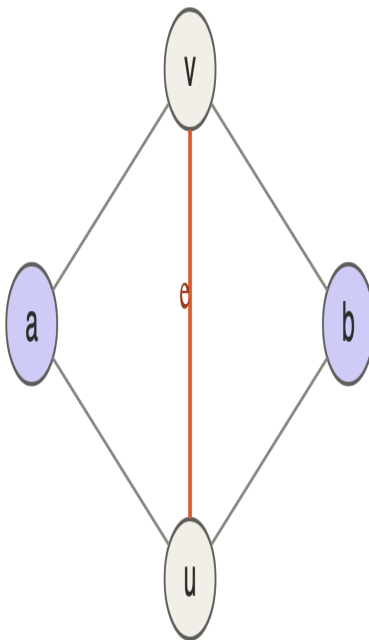


Figure 6: G , with $e = uv$

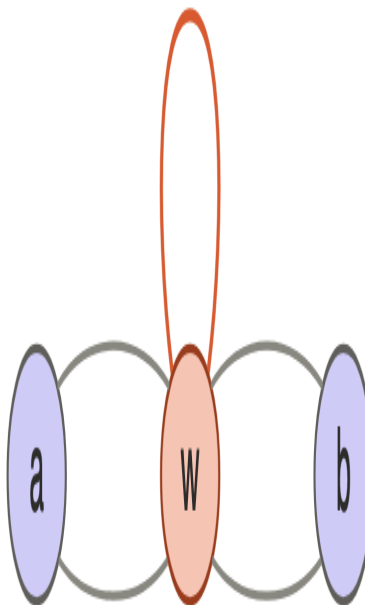


Figure 7: After identifying u and v as w

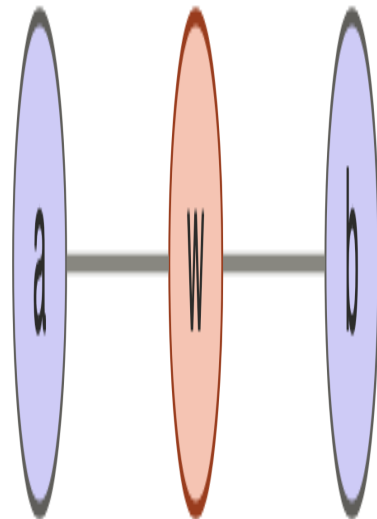


Figure 8: G/e , loops and parallel edges removed

Wagner-Kurotawski Theorem: These are two separate theorems, both of which establish that a graph is planar if and only if it does not contain a K_5 or $K_{3,3}$ minor, and does not contain a subgraph that is a subdivision of K_5 or $K_{3,3}$. $K_{3,3}$ is shown by Figure 4 and K_5 , the complete graph on 5 vertices, is drawn below for reference, remember a planar graph is one that can be rearranged on a flat surface such that no edges cross over each other.

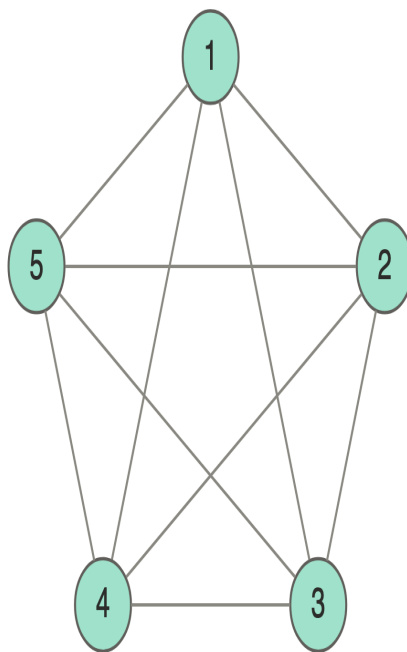


Figure 9: K_5 , the complete graph on five vertices

NP-completeness: A problem is in the NP class, if a solution can be verified in polynomial time ($O(n^c)$, where c is a constant). A problem is in the subclass P, if it can be solved in polynomial time. Whether all NP problems can be solved in polynomial time is an open question. An NP-Hard problem is one where there is no problem in the NP class strictly harder than it (Note: this does not mean that the problem is in NP, it could take longer than polynomial time to verify a solution). An NP-Complete problem is one that is both NP-Hard and in NP.

Polynomial-time Reductions: A reduction is a way to solve for the algorithmic complexity of a problem, say A, by transforming it into another problem B, whose complexity we do know. For our purposes, we will be using polynomial-time reductions, which means that the transformation from A to B must be done in polynomial time. If we can show that a problem A can be reduced to a problem B, then we know that A is no harder than B. If we can show that a problem B can be reduced to a problem A, then we know that B is no harder than A. If we can show both, then we know that A and B are of equal complexity.

Dynamic programming on trees: Dynamic programming solves a large problem by breaking it into subproblems, computing each one exactly once, and storing the result. It pays off when each subproblem can be summarized by a small amount of information, called a *state*, so that the table of stored results stays manageable even when the number of possible solutions becomes enormous.

Trees are unusually well suited to this. Because a tree contains no cycles, deleting a single vertex disconnects it. So if we choose an arbitrary root and write T_v for the subtree hanging below a vertex

v , then T_v meets the rest of the tree only at v itself. Nothing inside T_v is visible from outside except through that one vertex.

This is what makes a small summary sufficient. For each vertex we record the best value achievable inside its subtree, once for each state of that vertex, and then we discard the subtree entirely. Working upward from the leaves, every vertex computes its values from the values already stored at its children. At the root we combine the stored states according to what the problem asks for: the maximum, the minimum, or simply whether any valid option survives at all.

3 Intro to Conflict-Free Coloring

Definition 3.1 (Conflict-free coloring). A **partial k -coloring** of G is an assignment $\chi : V' \rightarrow \{1, \dots, k\}$ for some subset $V' \subseteq V(G)$; vertices outside V' are **uncolored**. Such a χ is **conflict-free** with respect to closed neighborhoods if for every $v \in V(G)$ there is a vertex $w \in N[v] \cap V'$ whose color is unique in $N[v]$ — that is, $\chi(w') \neq \chi(w)$ for every other $w' \in N[v] \cap V'$. We call w the **conflict-free neighbor** of v . The **conflict-free chromatic number** $\chi_{CF}(G)$ is the smallest k for which such a coloring exists.

Two key notes to keep in mind. First, the coloring is *partial*: leaving vertices uncolored is permitted, and is often necessary. Second, an uncolored vertex still needs a conflict-free neighbor — being uncolored exempts v from carrying a color, not from satisfying the condition.

The all-vertices variant, in which every vertex must be colored, is also studied in this paper. The two never differ by more than one color, since a fresh color can be handed to whatever was left uncolored. However, some of the paper's tight results don't work without partial coloring.

Throughout, **orange** is color 1, **green** is color 2, and a **hollow** vertex is uncolored. Uncolored vertices are permitted — see the remark on partial colorings below — and are drawn hollow rather than omitted, since they still belong to the neighborhoods being checked.

3.1 A single vertex: $\chi_{CF} = 1$

For $K_1 = \{v\}$ we have $N[v] = \{v\}$. Coloring v with color 1 makes that color occur exactly once in $N[v]$, so one color suffices, and zero colors leave $N[v]$ with no colors at all. Hence $\chi_{CF}(K_1) = 1$.

For K_2 on $\{u, v\}$, color u and leave v uncolored. Then $N[u] = N[v] = \{u, v\}$ contains color 1 exactly once, so both vertices are satisfied by u . Note that coloring *both* vertices 1 would fail — this is the first sign that adding color can destroy a conflict-free coloring, which is why the definition permits uncolored vertices.

For the path $a - b - c$, color the middle vertex b . Then $N[a] = \{a, b\}$, $N[b] = \{a, b, c\}$, and $N[c] = \{b, c\}$ all contain b and no other colored vertex, so b is the conflict-free witness for all three.

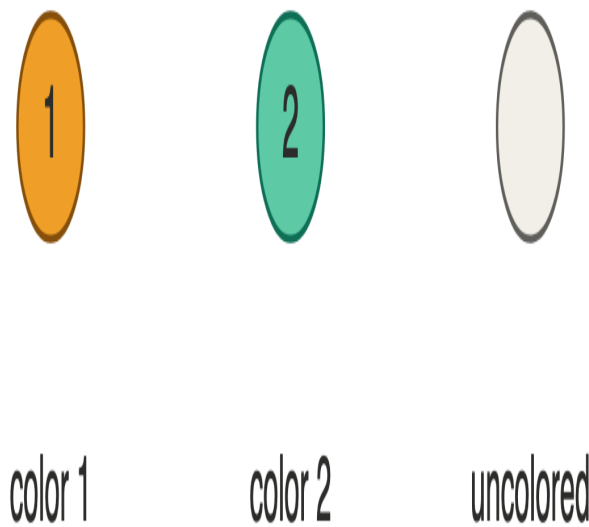


Figure 10: Color convention used in every figure below



Figure 11: K_1 : $\chi_{CF} = 1$

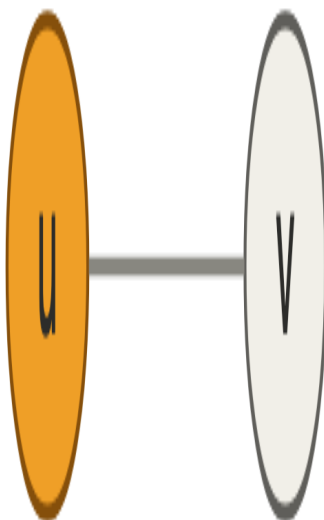


Figure 12: K_2 : $\chi_{CF} = 1$

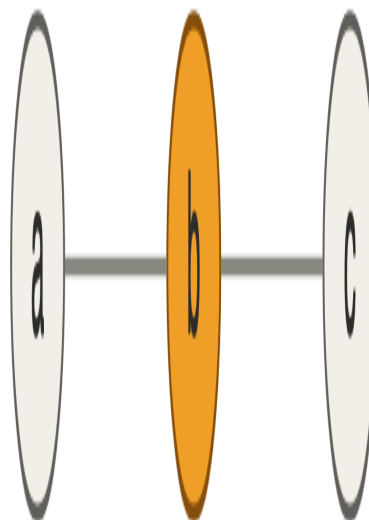


Figure 13: P_3 : $\chi_{CF} = 1$

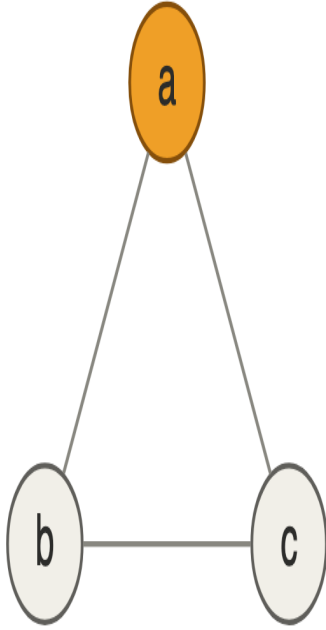


Figure 14: $C_3 = K_3$: $\chi_{CF} = 1$

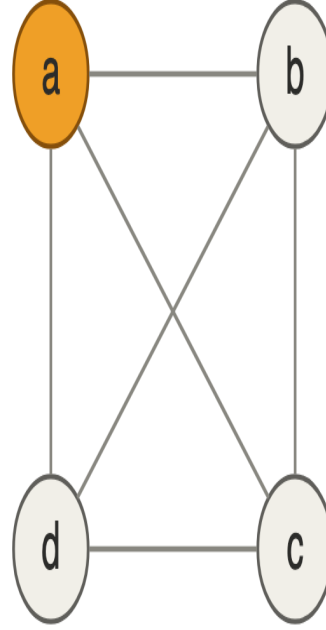


Figure 15: K_4 : $\chi_{CF} = 1$, while $\chi(K_4) = 4$

3.2 Complete graphs: $\chi_{CF}(K_n) = 1$

Both follow from one observation. In K_n every vertex is adjacent to every other, so $N[v] = V$ for all v : there is only one closed neighborhood to check. Color a single vertex and leave the remaining $n - 1$ uncolored; that color occurs exactly once in V , and every vertex sees it. So $\chi_{CF}(K_n) = 1$ for all $n \geq 1$.

More generally, the same argument gives $\chi_{CF}(G) = 1$ whenever G has a **universal vertex** — a vertex adjacent to all others — since coloring it alone witnesses every closed neighborhood. This covers K_n , stars, and wheels at once.

Conflict-free vs. proper coloring. For closed neighborhoods the two notions are related in one direction, and the relation is easy to miss: *every proper coloring is already conflict-free*. In a proper coloring no neighbor of v shares v 's color, so that color occurs exactly once in $N[v]$ and v serves as its own conflict-free neighbor. Hence

$$\chi_{CF}(G) \leq \chi_P(G) \quad \text{for every graph } G,$$

writing χ_P for the ordinary chromatic number.

The inequality can be arbitrarily loose, and K_4 is the cleanest witness: properly coloring it needs four colors, since every pair of vertices is adjacent, while conflict-free coloring needs one. In general $\chi_P(K_n) = n$ and $\chi_{CF}(K_n) = 1$, so the gap grows without bound.

None of this survives the move to open neighborhoods, where $v \notin N(v)$ and the “ v is its own witness” argument evaporates. There the two parameters are incomparable: no constant c satisfies $c \cdot \chi_O(G) \leq \chi_P(G)$ for all G , and none satisfies $c \cdot \chi_P(G) \leq \chi_O(G)$ either. So the intuition that conflict-free coloring is the easier problem is specific to closed neighborhoods.

3.3 $K_{2,3}$: the first jump to two colors

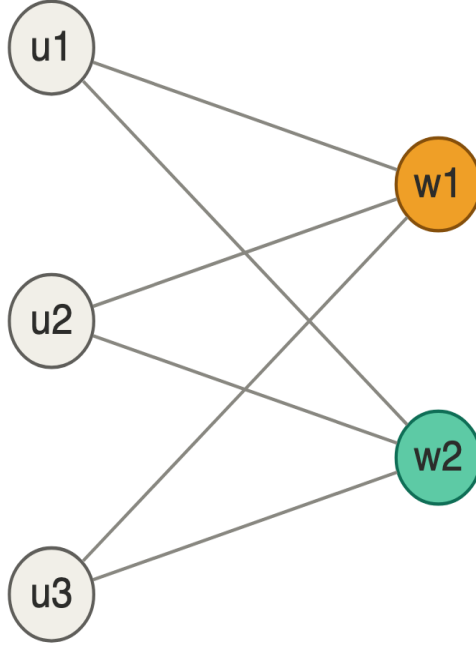


Figure 16: $K_{2,3}$ with the 2-side colored: $\chi_{CF} = 2$

Write $U = \{u_1, u_2, u_3\}$ for the side of size three and $W = \{w_1, w_2\}$ for the side of size two, so that

$$N[u_i] = \{u_i, w_1, w_2\}, \quad N[w_j] = \{w_j, u_1, u_2, u_3\}.$$

Two colors suffice. Color w_1 with 1 and w_2 with 2, leaving U uncolored. Each $N[u_i]$ then contains one vertex of each color, so both are unique. $N[w_1]$ contains only w_1 colored, and $N[w_2]$ only w_2 . Every closed neighborhood has a uniquely occurring color, so $\chi_{CF}(K_{2,3}) \leq 2$.

One color does not. Since $N[u_i]$ depends only on u_i and the two vertices of W , split on how many of w_1, w_2 are colored. The three cases are exhaustive. In each figure below, the double-circled vertex is the one left without a witness.

Neither w_j colored. Then $N[u_i] = \{u_i, w_1, w_2\}$ can only be satisfied by u_i itself, so all three of u_1, u_2, u_3 are colored 1. But $N[w_1] \supseteq U$ now contains color 1 three times and w_1 is uncolored, so w_1 has no uniquely occurring color.

Exactly one w_j colored, say w_1 . Every $N[u_i]$ already contains w_1 , so coloring u_i would make color 1 appear twice there; hence all of U is uncolored. But then $N[w_2] = \{w_2\} \cup U$ contains no colored vertex at all.

Both w_j colored. Then color 1 appears at least twice in every $N[u_i] \supseteq \{w_1, w_2\}$, and coloring u_i only raises the count to three. So no u_i has a uniquely occurring color in its closed neighborhood.

Every case fails, so $\chi_{CF}(K_{2,3}) \geq 2$ and therefore $\chi_{CF}(K_{2,3}) = 2$.

In each earlier example some single vertex was visible from everywhere, so coloring it alone settled every neighborhood at once. $K_{2,3}$ is the first graph here without such a vertex, and the middle case

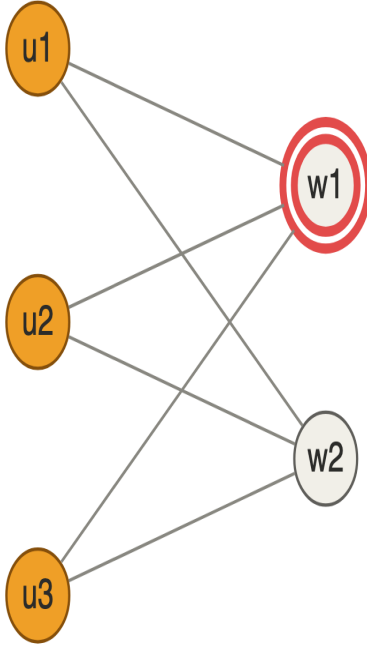


Figure 17: Neither w_j colored

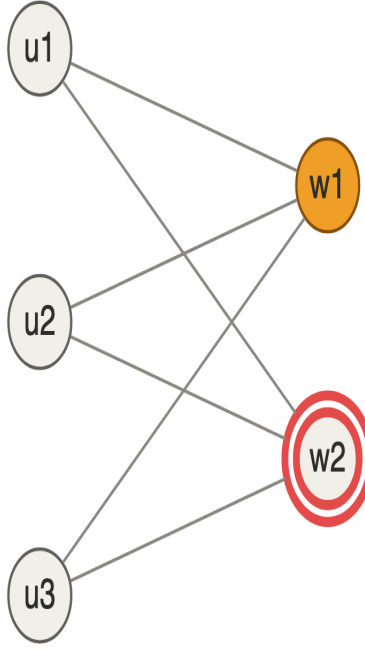


Figure 18: Only w_1 colored

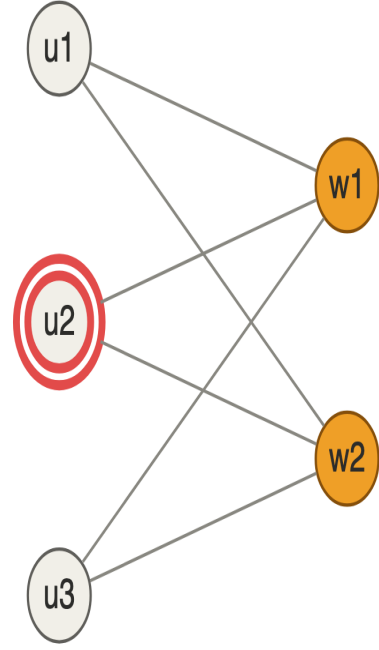


Figure 19: Both w_j colored

shows what breaks: w_1 and w_2 are non-adjacent and share all their neighbors, so no single colored vertex can serve as the witness for both.

Note that $K_{2,3}$ is not the smallest graph needing two colors — C_4 needs two on only four vertices, and the argument is the same, since the two diagonal pairs each have identical closed neighborhoods. And having no universal vertex is *sufficient for nothing*: the path on $3n$ vertices has no universal vertex and still has $\chi_{CF} = 1$, by coloring the middle vertex of each consecutive triple. What $K_{2,3}$ and C_4 add beyond the absence of a universal vertex is the *competition* — two vertices with the same closed neighborhood, each needing a witness the other cannot supply.

4 Proof 1 - Necessary Coloring

4.1 Lemma 3.2 - The Forcing Gadget

Let $N(v)$ be an open neighborhood, meaning vertex v isn't in its own neighborhood. Let $N[v] = N(v) \cup \{v\}$ be the closed neighborhood, which is the one the conflict-free condition is checked on.

Take two disjoint, independent copies of graph H , with $\chi_{CF}(H) = k$, both connected to v and to nothing else. Then v must be colored, because:

For the sake of Contradiction, suppose v is uncolored. Then each copy of H has to be conflict-free colored on its own, using k colors. We have two H -graphs, and every color appears at least twice in $N(v)$, meaning nothing in $N[v]$, the closed neighborhood, is unique, so v has no conflict-free neighbor. This is a contradiction, so v must be colored.

If additionally vertex u is adjacent to v , and $N(u) \cap N(v)$ contains two copies of graph J with $\chi_{CF}(J) = k - 1$, connected to nothing but u and v . Without loss of generality, assume u and v had color 1. Then we'd have to color J with $k - 1$ colors, and each color would appear at least twice, and color 1 appears on both v and u , meaning u and v see every color twice, so neither has a conflict-free neighbor. This is a contradiction, so u and v must have distinct colors.

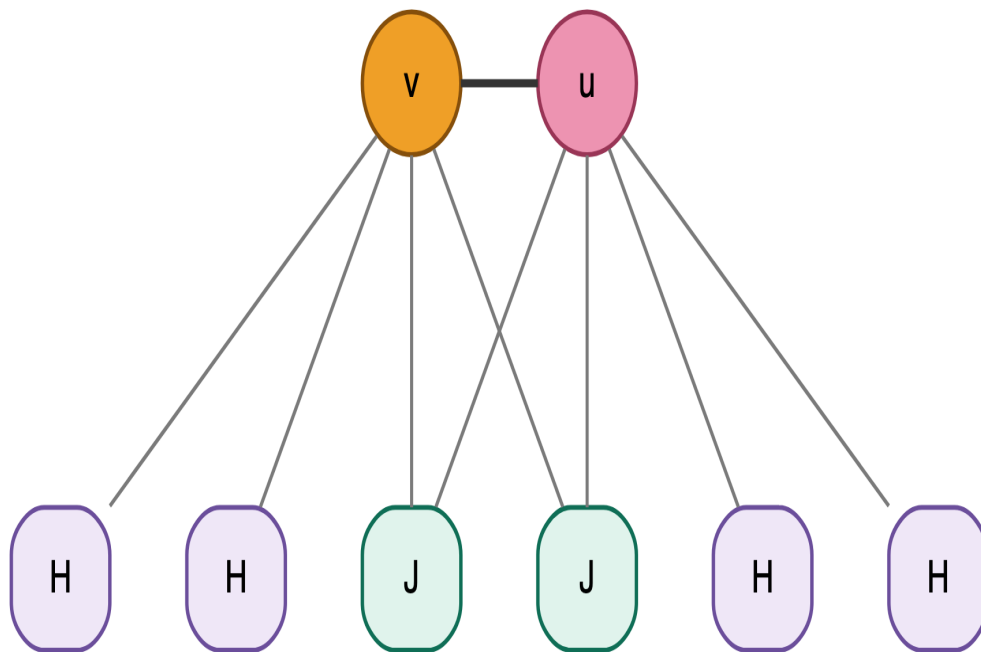


Figure 20: The configuration of Lemma 3.2. The two J copies are the only boxes joined to both u and v .

Note that for this lemma, $\chi_{CF}(J)$ doesn't have to equal $k - 1$, we can do the same thing, if $\chi_{CF}(J) = k$. However, the J being $k - 1$ colorable will prove useful later, particularly in allowing us to construct a recursion.

Note that because u and v are colored, H and J can be uncolored because every vertex in H is adjacent to v , which has a distinct color, and the same applies to J where u and v have distinct colors.

4.2 Lemma 3.3 - The G_K Hierarchy

Note we're working with $N[\]$, the closed neighborhood.

4.2.1 Why we need this

To use Lemma 3.2, you need graphs with $\chi_{CF} = k$. We have to build them recursively.

4.2.2 Base cases

G_1 is a single vertex, and $\chi_{CF}(G_1) = 1$.

$G_2 = K_{1,3}$ with the edge cD subdivided, where $K_{1,3}$ is the complete bipartite graph with one vertex on one side and 3 on the other.

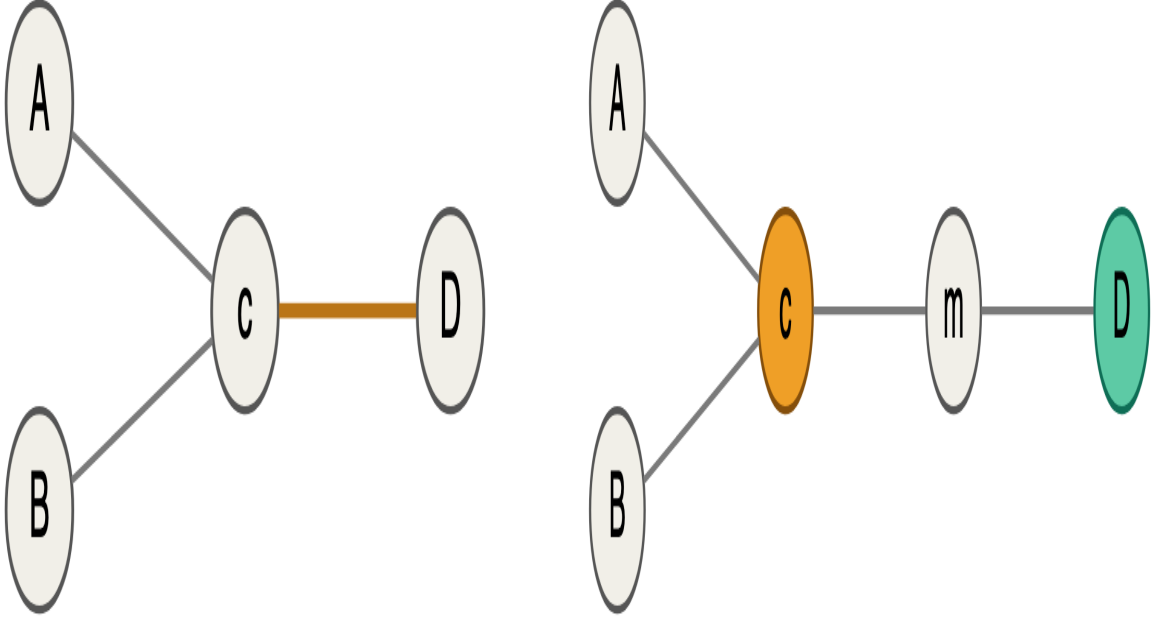


Figure 21: $K_{1,3}$, with the edge cD highlighted Figure 22: G_2 : the same graph with cD subdivided by m

$\chi_{CF}(G_2) = 2$, because with one color there are only two cases and both fail:

- if c and D are colored, then $N[m]$ doesn't satisfy conflict-free coloring;
- if A and B are colored, then $N[c]$ doesn't satisfy conflict-free coloring.

Note that the paper is using Closed neighborhoods. The base cases in the recursive algorithm of Lemma 3.3, don't work with open neighborhoods. If we take a single, isolated vertex, u , then this vertex is uncolorable on an open neighborhood; no matter how many k -colors you have, there are no vertices adjacent to u that have a unique color, and we can't count u by the definition of open neighborhood.

4.2.3 The recursive step

Recursion proof. Assume we have $\chi_{CF}(G_k) = k$ and $\chi_{CF}(G_{k-1}) = k - 1$.

- Take a complete graph $G = K_{k+1}$ on $k + 1$ vertices.

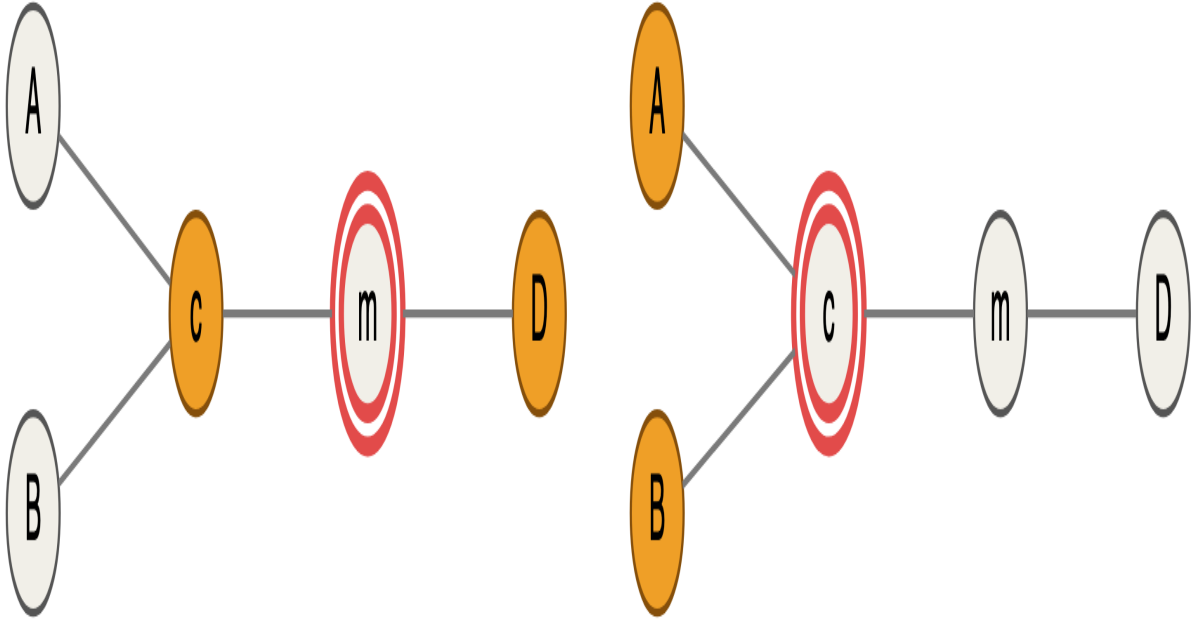


Figure 23: Case 1: c colored, so m has no witness Figure 24: Case 2: c uncolored, so c has no witness

- To each vertex $v \in V(K_{k+1})$ attach two disjoint and independent copies of G_k , adding an edge from every v to every vertex of both copies of G_k .
- For each edge $e = vw \in E(K_{k+1})$, add two disjoint and independent copies of G_{k-1} , where there is an edge from every v and w to every vertex on both copies of G_{k-1} .

For G_{k+1} where $k + 1 = 3$, K_3 is the base.

4.2.4 Lower bound

Now we can apply Lemma 3.2. Let $G_2 = H$ and $G_1 = J$. Then we know that every pair of vertices in K_3 must have a different color, so no two vertices can share a color. At minimum we need 3 colors for a conflict-free coloring.

We generalize, and by induction there exists G_k for all k . Since the central graph is K_k and every pair of vertices is adjacent, every such graph must have at least k colors; in other words $\chi_{CF}(G_k) \geq k$.

Note that ruling out k -colorings also rules out every smaller number of colors, since a conflict-free k -coloring may draw from $\{1, \dots, k\}$ without using them all.

4.2.5 Upper bound

It remains to show $\chi_{CF}(G_{k+1}) \leq k + 1$, which together with the lower bound gives $\chi_{CF}(G_{k+1}) = k + 1$.

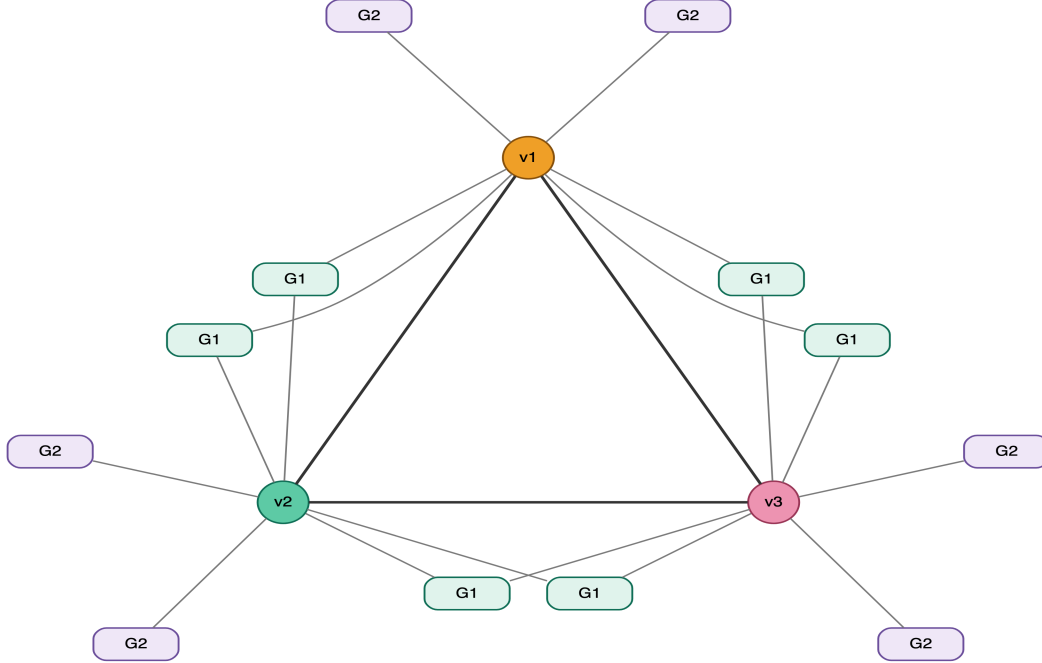


Figure 25: G_3 : a K_3 core, two copies of G_2 on each vertex, two copies of G_1 on each edge.

Claim. Color the $k + 1$ vertices of the core K_{k+1} with distinct colors $1, \dots, k + 1$ and leave *every other vertex uncolored*. This is a conflict-free $(k + 1)$ -coloring of G_{k+1} .

Proof. There are three kinds of vertex to check.

Core vertices. Let v_i be a core vertex. The only colored vertices anywhere in $N[v_i]$ are the core vertices, and they carry pairwise distinct colors. So the color of v_i occurs exactly once in $N[v_i]$, and v_i is its own conflict-free neighbor.

Vertices in a vertex gadget. Let x lie in a copy of G_k attached to v_i . By construction that copy is adjacent to no vertex of G_{k+1} other than v_i , so $N[x] \subseteq V(\text{copy}) \cup \{v_i\}$. The copy is uncolored, so the color of v_i occurs exactly once in $N[x]$, and v_i is the conflict-free neighbor of x .

Vertices in an edge gadget. Let y lie in a copy of G_{k-1} attached to the edge $v_i v_j$. Then $N[y] \subseteq V(\text{copy}) \cup \{v_i, v_j\}$, and the copy is uncolored, so exactly two colors appear in $N[y]$, each once. Either one serves as the conflict-free neighbor of y .

Every vertex has a conflict-free neighbor, so the coloring is valid and $\chi_{CF}(G_{k+1}) \leq k + 1$. ■

Combining, $\chi_{CF}(G_k) = k$ for all $k \geq 1$.

Two things worth noticing about this coloring. First, it colors only $k + 1$ vertices out of the whole of G_{k+1} , which is why the same construction is useful when studying the conflict-free domination number γ_{CF}^k . Second, leaving the gadgets uncolored is exactly what makes the argument consistent with Lemma 3.2: the “sealed off” reasoning there applies only when the attachment vertex is uncolored, and here it is colored.

4.2.6 Counting the vertices

$$|G_1| = 1, \quad |G_2| = 5, \quad |G_3| = 3 + 3(2|G_2|) + 3(2|G_1|) = 3 + 30 + 6 = 39.$$

In general, the core contributes $k + 1$ vertices, each of the $k + 1$ core vertices carries $2|G_k|$, and each of the $\binom{k+1}{2}$ core edges carries $2|G_{k-1}|$. Since

$$\binom{k+1}{2} = \frac{(k+1)!}{2!(k-1)!} = \frac{k(k+1)}{2},$$

we get

$$|G_{k+1}| = \underbrace{(k+1)}_{v \in K_{k+1}} + \underbrace{2(k+1)|G_k|}_{H \text{ graphs}} + \underbrace{k(k+1)|G_{k-1}|}_{J \text{ graphs}}$$

and factoring out $(k+1)$,

$$\boxed{|G_{k+1}| = (k+1) \left(2|G_k| + k|G_{k-1}| + 1 \right).$$

Check: $|G_3| = 3(2 \cdot 5 + 2 \cdot 1 + 1) = 3 \cdot 13 = 39$ and $|G_4| = 4(2 \cdot 39 + 3 \cdot 5 + 1) = 4 \cdot 94 = 376$.

4.2.7 Algorithmic bounds

Every term of the bracket is positive, so $a_{k+1} \geq 2a_k$, and therefore

$$a_k = \Omega(2^k).$$

This is the bound stated in the paper, and it is all that is needed there. We can do better by keeping the factor in front: since $a_{k+1} \geq 2(k+1)a_k$, unrolling from $a_2 = 5$ gives

$$a_k \geq 5 \prod_{j=3}^k 2j = 5 \cdot 2^{k-2} \cdot \frac{k!}{2},$$

so $\log_2 a_k = \Omega(k \log k)$.

The sequence is increasing, so $a_{k-1} \leq a_k$, and also $1 \leq a_k$. Hence

$$a_{k+1} \leq (k+1)(2a_k + k a_k + a_k) = (k+1)(k+3) a_k \leq (k+3)^2 a_k.$$

Unrolling from $a_1 = 1$,

$$a_{k+1} \leq \prod_{j=1}^k (j+3)^2 \leq (k+3)^{2k},$$

and taking logarithms,

$$\log_2 a_{k+1} \leq 2k \log_2(k+3) = O(k \log k),$$

so $a_k = \mathcal{O}(2^{k \log k})$.

4.2.7.1 Solving the recurrence

First we rewrite our recurrence as a sequence:

$$a_1 = 1, \quad a_2 = 5, \quad a_{k+1} = (k+1)(2a_k + k a_{k-1} + 1), \quad k, a \in \mathbb{Z}^+.$$

The $(k+1)$ out front and the k on the second term hide a Pell recurrence. We divide it out.

4.2.8 Normalizing away the factorials

Divide both sides by $(k+1)!$:

$$\frac{a_{k+1}}{(k+1)!} = \frac{2a_k + k a_{k-1} + 1}{k!}.$$

Let $b_k = \frac{a_k}{k!}$. The middle term simplifies because $\frac{k a_{k-1}}{k!} = \frac{a_{k-1}}{(k-1)!} = b_{k-1}$, so

$$b_{k+1} = 2b_k + b_{k-1} + \frac{1}{k!}.$$

Isolating the $1/k!$, what is left is exactly the **Pell recurrence**

$$b_{k+1} = 2b_k + b_{k-1}.$$

The seeds come from the graph: $b_1 = \frac{a_1}{1!} = 1$, where a_1 is the number of vertices in G_1 , and

$b_2 = \frac{a_2}{2!} = \frac{5}{2}$, referencing $K_{1,3}$ for a_2 .

4.2.9 The Pell recurrence

$$P_n = 2P_{n-1} + P_{n-2}, \quad P_0 = 0, P_1 = 1, \quad \langle 0, 1, 2, 5, 12, 29, 70, \dots \rangle$$

Set $P_n = r^n$, so $r^n = 2r^{n-1} + r^{n-2}$, giving $r^2 = 2r + 1$ and

$$r^2 - 2r - 1 = 0, \quad r = \frac{2 \pm \sqrt{4+4}}{2} = \frac{2 \pm \sqrt{8}}{2}, \quad r = 1 + \sqrt{2}, r = 1 - \sqrt{2}.$$

The recurrence is linear, so

$$P_n = A(1 + \sqrt{2})^n + B(1 - \sqrt{2})^n$$

where A and B satisfy all values of the recurrence. From $P_0 = 0$ we get $A = -B$, and from $P_1 = 1$,

$$1 = A(1 + \sqrt{2}) - A(1 - \sqrt{2}) = A(1 + \sqrt{2} + \sqrt{2} - 1) = A \cdot 2\sqrt{2},$$

so $A = \frac{1}{2\sqrt{2}}$ and $B = \frac{-1}{2\sqrt{2}}$, giving

$$P_n = \frac{(1 + \sqrt{2})^n - (1 - \sqrt{2})^n}{2\sqrt{2}}.$$

4.2.10 The homogeneous part

Running the Pell recurrence forward from b_1, b_2 :

$$b_3 = 2b_2 + b_1, \quad b_4 = 2b_3 + b_2 = 5b_2 + 2b_1, \quad b_5 = 2b_4 + b_3 = 12b_2 + 5b_1.$$

The coefficients are Pell numbers, so we generalize:

$$b_k = P_{k-1} b_2 + P_{k-2} b_1 = \frac{5}{2}P_{k-1} + P_{k-2}.$$

4.2.11 The inhomogeneous part

Our actual recurrence has $1/k!$ added, so

$$b_3 = 6 + \frac{1}{2!} = 6.5, \quad b_4 = 2b_3 + b_2 + \frac{1}{3!} = 5b_2 + 2b_1 + \frac{1}{3!} + 2\left(\frac{1}{2!}\right),$$

$$b_5 = 2b_4 + b_3 + \frac{1}{4!} = 12b_2 + 5b_1 + 2\left(\frac{1}{3!}\right) + 5\left(\frac{1}{2!}\right) + \frac{1}{4!}.$$

Each factorial creates its own Pell: a term $\frac{1}{j!}$ injected at step j propagates forward with Pell coefficients, contributing $\frac{P_{k-j}}{j!}$ to b_k .

Verification. Let $k = 5, j = 2$. We have $\frac{P_3}{2!}$, where $P_3 = 5$, so $\frac{5}{2!}$ — which is the coefficient appearing on $\frac{1}{2!}$ in b_5 above.

Accounting for every j , for some k :

$$\sum_{j=2}^{k-1} \frac{P_{k-j}}{j!}.$$

4.2.12 The closed form

Solving for our altered Pell recurrence:

$$b_k = \frac{5P_{k-1}}{2} + P_{k-2} + \sum_{j=2}^{k-1} \frac{P_{k-j}}{j!}.$$

Remember that $b_k = \frac{a_k}{k!}$, and that a_k is just our actual recurrence sequentialized, so

$$\boxed{|G_k| = k! \left[P_{k-2} + \frac{5P_{k-1}}{2} + \sum_{j=2}^{k-1} \frac{P_{k-j}}{j!} \right]}$$

with P_n as above. Using the standard backward extension $P_{-1} = 1$, this is valid for all $k \geq 1$.

Check. $k = 3$ gives $6 \left[2 + \frac{5}{2} + \frac{P_1}{2!} \right] = 6(6.5) = 39$, and $k = 4$ gives $24 \left[5 + \frac{25}{2} + \frac{P_2}{2!} + \frac{P_1}{3!} \right] = 24 \left(\frac{47}{3} \right) = 376$, matching the direct counts. The sequence continues 1, 5, 39, 376, 4545, 65826, 1112461, ...

4.2.13 Growth rate

Write $\alpha = 1 + \sqrt{2}$ and $\beta = 1 - \sqrt{2}$, so

$$P_n = \frac{\alpha^n - \beta^n}{2\sqrt{2}}.$$

Since $|\beta| = \sqrt{2} - 1 < 1$, the β^n term vanishes and

$$\lim_{n \rightarrow \infty} P_n = \lim_{n \rightarrow \infty} \frac{\alpha^n}{2\sqrt{2}}, \quad \text{so} \quad P_n = \Theta(\alpha^n) = \Theta((1 + \sqrt{2})^n).$$

For the sum, $P_{k-j} = \Theta(\alpha^{k-j})$, so

$$\sum_{j=2}^{k-1} \frac{P_{k-j}}{j!} = \Theta\left(\sum_{j=2}^{k-1} \frac{\alpha^k}{j! \alpha^j}\right) = \Theta\left(\alpha^k \sum_{j=2}^{k-1} \frac{1}{j! \alpha^j}\right),$$

where $\sum_{j \geq 2} \frac{1}{j! \alpha^j}$ converges — to $e^{1/\alpha} - 1 - \frac{1}{\alpha}$ — so the whole sum is $\Theta(\alpha^k)$. (The $j = 2$ term alone is already $\Theta(\alpha^k)$, which gives the matching lower bound.)

The remaining pieces $\frac{5P_{k-1}}{2}$ and P_{k-2} are Pell recurrences multiplied by the constants $\frac{5}{2\alpha}$ and $\frac{1}{\alpha^2}$ respectively, so they are both $\Theta(\alpha^k)$ as well.

So $\frac{|G_k|}{k!} = \Theta((1 + \sqrt{2})^k)$, and thus

$$\boxed{|G_k| = \Theta(k! (1 + \sqrt{2})^k)}.$$

Because every piece has the same growth constant, the ratio actually converges rather than merely being bounded:

$$\lim_{k \rightarrow \infty} \frac{|G_k|}{k! \alpha^k} = \frac{1}{2\sqrt{2}} \left(e^{\sqrt{2}-1} + \frac{1-\sqrt{2}}{2} \right) \approx 0.4617667,$$

using $1/\alpha = \sqrt{2} - 1$.

4.2.14 Comparison with the paper

The paper states only that $|G_k|$ is in $\Omega(2^k)$ and $\mathcal{O}(2^{k \log k})$, for **fixed** k the gadget has constant size, which is what makes the reduction in Lemma 3.4 run in polynomial time. The exact rate is consistent with both bounds, since $\log_2(k! \alpha^k) = \Theta(k \log k)$, but the closed form derived in this document along with its growth rate are strictly sharper because we pin the base at $1 + \sqrt{2} \approx 2.414$

4.3 Lemma 3.4 — The Reduction

Goal: Show that k -coloring can be transformed into conflict-free k -coloring using a polynomial-time reduction. That is,

$$k\text{-Coloring} \preceq_p \text{CF-Coloring}.$$

Take a graph G , for which we want to check whether we can achieve a proper coloring with k colors, where k is fixed. We're going to try to construct a graph G' by transforming G such that

$$G \text{ is properly } k\text{-colorable} \iff G' \text{ is CF } k\text{-colorable}.$$

Achieving this would let us solve ordinary k -coloring if we can solve CF-coloring.

4.3.1 The construction

We can generalize the Lemma 3.3 results by taking some arbitrary graph G that can be properly colored with k colors.

For every $v \in V(G)$, attach two copies of G_k to each vertex, where G_k is our gadget graph, adding an edge from v to every vertex of both copies. For every edge $e = uv \in E(G)$, attach both u and v to two copies of G_{k-1} , adding an edge from u and from v to every vertex of both copies. Call the result G' .

Note the difference: we are no longer working with complete graphs as the base of our construction, but we know this works for all k colors. The construction needs both G_k and G_{k-1} to exist, so it requires $k \geq 2$.

4.3.2 Direction 1: From χ_P of G to χ_{CF} of G'

Now color the subgraph G of G' as if it were being properly colored with k colors, and leave all the gadget vertices uncolored.

All the vertices of every copy of G_k are only adjacent to one colored vertex of G , namely the v that copy hangs off, so the color of v occurs exactly once in their closed neighborhoods and a conflict-free coloring is achieved.

Vertices which share an edge (i.e. all uv) have different colors, because we colored G properly, meaning that every vertex in all copies of G_{k-1} adjacent to both u and v sees exactly two colors, each once, and so is also conflict-free colorable.

For the original vertices, because we are operating in the closed neighborhood $N[v]$, v is its own conflict-free neighbor: no neighbor of v in G carries v 's color, since the coloring of G is proper, and every gadget vertex in $N[v]$ is uncolored.

So G' can be conflict-free colored with k colors.

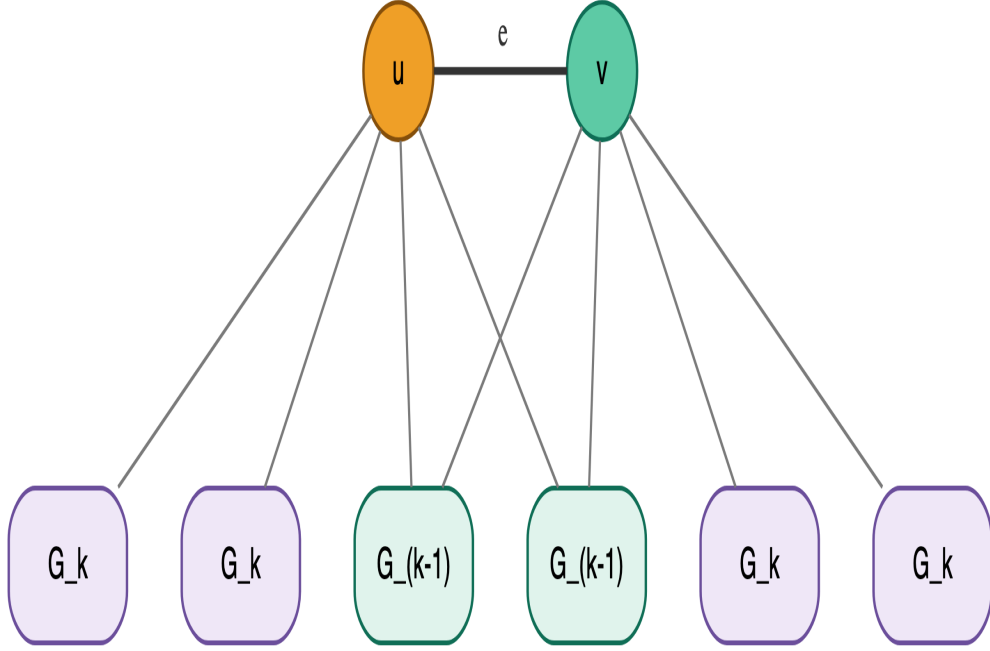


Figure 26: The reduction applied to a single edge uv of G . The gadgets play the roles of H and J from Lemma 3.2.

4.3.3 Direction 2: From a χ_{CF} of G' to a χ_P of G

Now we try to construct G from some G' .

Suppose G' has some conflict-free k -coloring. Then all the vertices of the subgraph G must be colored, per Lemma 3.2: each $v \in V(G)$ has two disjoint independent copies of G_k in its open neighborhood, attached to nothing else, and $\chi_{CF}(G_k) = k$.

The endpoints of every edge must differ, because $\chi(u) \neq \chi(v)$ by the second part of Lemma 3.2: the two copies of G_{k-1} attached to uv lie in $N(u) \cap N(v)$, are attached to nothing else, and satisfy $\chi_{CF}(G_{k-1}) = k - 1$.

A proper coloring of G requires all vertices be colored and that adjacent vertices not share a color. We just proved both, so G is properly colorable with k colors.

For all $k \geq 3$ we know proper coloring is NP-complete. Lemma 3.4 shows that

$$\text{Proper } k\text{-Coloring} \preceq_p \text{Conflict-Free Coloring}.$$

The reduction itself is valid for every $k \geq 2$; it only yields hardness from $k = 3$ upward, because proper 2-coloring is in P (a graph is properly 2-colorable iff it is bipartite, which is decidable by breadth-first search).

4.3.4 Verifying the Reduction is in P

Note that the reduction is in polynomial time because we are considering a fixed k . This means $|G_k|$ and $|G_{k-1}|$ are constant. Since each vertex of G carries two copies of G_k and each edge carries two copies of G_{k-1} , label the per-vertex and per-edge costs

$$C_1 := 2|G_k|, \quad C_2 := 2|G_{k-1}|,$$

both constants once k is fixed. Writing $n = |V(G)|$ and $m = |E(G)|$, our count becomes

$$|V(G')| = \underbrace{n}_{\text{original}} + \underbrace{2n|G_k|}_{\text{vertex gadgets}} + \underbrace{2m|G_{k-1}|}_{\text{edge gadgets}} = n(C_1 + 1) + C_2 m,$$

so $|V(G')| = \mathcal{O}(n + m)$.

The edges come out the same way. Each vertex gadget contributes its own $|E(G_k)|$ internal edges plus $|G_k|$ join edges, and each edge gadget contributes $|E(G_{k-1})|$ internal edges plus $2|G_{k-1}|$ join edges, since the copy is joined to *both* endpoints. With $D_1 := 2(|G_k| + |E(G_k)|)$ and $D_2 := 2(2|G_{k-1}| + |E(G_{k-1})|)$,

$$|E(G')| = m + D_1 n + D_2 m = \mathcal{O}(n + m).$$

In other words we can construct our output graph G' in linear time.

Check. For $k = 3$ we have $|G_3| = 39$ with 69 edges and $|G_2| = 5$ with 4 edges, giving $C_1 = 78$, $C_2 = 10$, $D_1 = 216$, $D_2 = 28$. Applying the reduction to $G = K_4$ (so $n = 4$, $m = 6$) predicts $4(79) + 60 = 376$ vertices and $6 + 216(4) + 28(6) = 1038$ edges — which is exactly G_4 , as it must be, since G_4 is this construction run on K_4 .

Alternatively, the fact that k is fixed means that the number of vertices and edges we need to construct *per vertex and per edge of G* is fixed.

Every vertex v in the subgraph G has $x = \deg_G(v)$ internal edges and $y = C_1$ edges to gadget vertices. The quantity y is constant, but x is not — it varies with G and can be as large as $n - 1$. So the two have to be accounted for separately.

The gadget work at v is constant: we instantiate C_1 new vertices, copy $2|E(G_k)|$ internal edges, and add C_1 join edges. That is some constant C'_1 operations per vertex, so processing all n vertices costs $C'_1 n = \mathcal{O}(n)$.

The internal edges are not paid for per vertex but per edge. We can apply the same logic to every edge: given m edges with C'_2 operations per edge — copying the edge itself, instantiating C_2 vertices, and adding D_2 edges — the total is $C'_2 m = \mathcal{O}(m)$. Equivalently, if one insists on charging internal edges to vertices, the handshake lemma gives $\sum_v \deg_G(v) = 2m$, which is $\mathcal{O}(m)$ in aggregate even though no individual term is bounded.

The total runtime is $\mathcal{O}(n + m)$. This checks out because every gadget we copy has constant size, so copying each gadget takes $\mathcal{O}(1)$, and there are $2n$ gadget copies on vertices and $2m$ gadget copies on edges.

What “linear” is hiding. The runtime is linear in n and m , but the constant is $C_1 = 2|G_k|$, and from the closed form derived earlier, $|G_k| = \Theta(k!(1 + \sqrt{2})^k)$. So the reduction is linear for each *fixed* k and catastrophic in k : already $C_1 = 752$ at $k = 4$. This is why the paper is careful to say k is fixed, and why the result is a statement about CF k -Coloring for each constant k rather than about the problem with k given as part of the input.

4.3.5 Conclusion

We know the reduction runs in polynomial time, so we know conflict-free coloring for a fixed k is NP-hard. Checking a proposed conflict-free coloring can be done in polynomial time, for each vertex, tally the colors in $N[v]$ and look for a count of one, so the problem is in NP. Therefore

Conflict-Free Coloring is NP-complete for every fixed $k \geq 3$.

The cases $k = 1$ and $k = 2$ are not settled by this argument, since proper 1- and 2-coloring are both in P. The paper handles them separately, by reduction from Positive Planar 1-in-3-SAT, and gets the stronger result that they are NP-complete even when the input is restricted to planar graphs.

4.4 Theorem 3.5 — A Sufficient Criterion

Goal. Prove that a graph with no K_{k+2} minor and no K_{k+3}^{-3} minor is conflict-free colorable with k colors, and that the coloring can be found in polynomial time by *iterated elimination of distance-3 sets*.

4.4.1 Notation for this section

Everything below uses the following symbols, and nothing else. They are listed here because the proof runs three separate arguments that would otherwise reuse letters.

Symbol	Meaning
$\mathcal{P}(k)$	the proposition being proved by induction on k
$\Delta(G)$	the maximum degree of G
$C_\ell = t_1 t_2 \cdots t_\ell t_1$	a cycle of length ℓ , with vertices t_i
T	a connected component of a forest, i.e. a tree
$P = v_0 v_1 \cdots v_L$	a longest path in T , with L edges
z, Q, u	a vertex off P ; a path from z to P ; the neighbor of v_i on Q
D	the set of vertices at pairwise distance ≥ 3 , colored in one round
$N[D]$	$\bigcup_{d \in D} N[d]$, the closed neighborhood of the whole set
W	the unseen set, $V(G) \setminus N[D]$
s	the seed : the vertex the algorithm puts into D first, chosen arbitrarily

Symbol	Meaning
U	an arbitrary set of unseen vertices, $U \subseteq W$
R	a connected component of $G \setminus U$
w, d, a, b	first vertex of R to join D ; a marked vertex outside R ; and the two interior vertices of the 3-edge path $w a b d$
y, f, x	an arbitrary vertex of W ; an arbitrary vertex outside W ; the vertex that $G \setminus W$ contracts to

4.4.2 The proposition

Let $\mathcal{P}(k)$ be the statement, quantified over **all** graphs:

$\mathcal{P}(k)$: for every graph X , if X has no K_{k+2} minor and no K_{k+3}^{-3} minor, then iterated elimination of distance-3 sets colors X with k colors.

We prove $\mathcal{P}(1)$ outright, then show that $\mathcal{P}(k-1) \Rightarrow \mathcal{P}(k)$ for every $k \geq 2$.

4.4.3 Base case: $k = 1$

At $k = 1$ the forbidden minors are $K_{k+2} = K_3$ and $K_{k+3}^{-3} = K_4^{-3}$. Recall that K_n^{-3} is K_n with the three edges of one triangle deleted; deleting the triangle on $\{1, 2, 3\}$ from K_4 leaves the edges $14, 24, 34$, so

$$K_4^{-3} = K_{1,3}.$$

Deleting the dashed triangle from K_4 leaves exactly the three edges at vertex 4, which is the star $K_{1,3}$.

Note that the two forbidden minors do exactly two different jobs: K_3 forbids cycles, and $K_{1,3}$ forbids branching. Together they force a disjoint union of paths, which we now prove in three steps.

4.4.4 Step 1: G has no K_3 minor, so it has no cycles

Proof by contradiction. Suppose G contains a cycle $C_\ell = t_1 t_2 t_3 \dots t_\ell t_1$. Because G is a simple graph, the length of the cycle is $\ell \geq 3$.

If $\ell = 3$, then $C_3 = K_3$ and we are done. If $\ell > 3$, then contract the edge $t_1 t_2$ into a new vertex t^* , so that t^* inherits the neighbors of both and the cycle becomes $t^* t_3 \dots t_\ell t^*$. We now have $\ell - 1$ vertices. Do this $\ell - 3$ times, so that we have $\ell - (\ell - 3) = 3$ vertices left, meaning we have C_3 .

We produced this graph through deletions and contractions, so it is a minor of C_ℓ and thus a minor of G . In both cases G has a K_3 subgraph or minor. By contradiction, G does not contain any cycle, and because G is acyclic it is a forest.

One detail worth checking: no parallel edge appears during a contraction, since a parallel edge would require t_1 and t_2 to share a neighbor, forcing $\ell = 3$, which is excluded in this case.

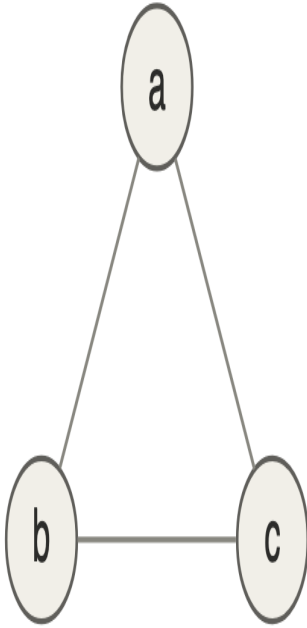


Figure 27: K_3

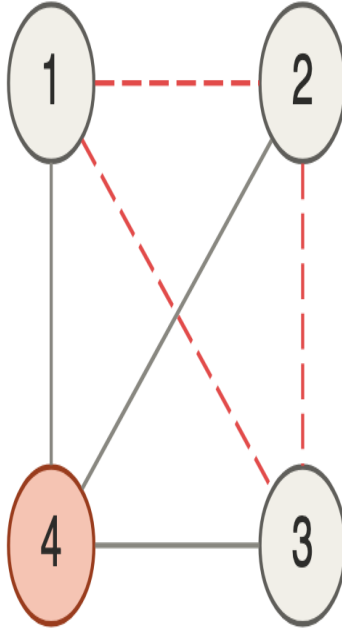


Figure 28: K_4 , triangle dashed

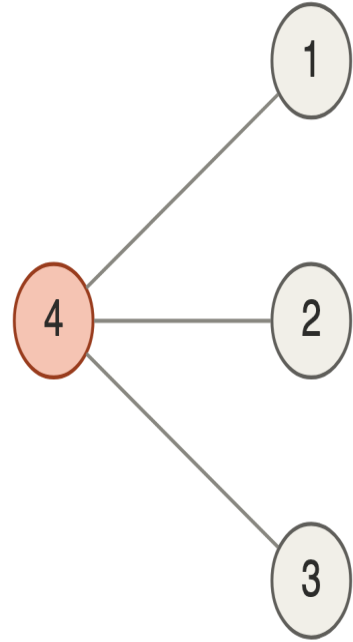


Figure 29: $K_4^{-3} = K_{1,3}$

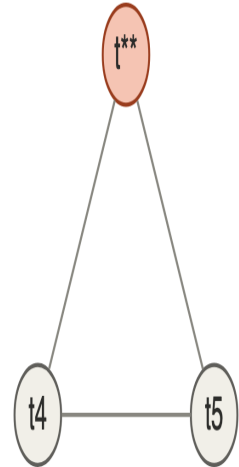
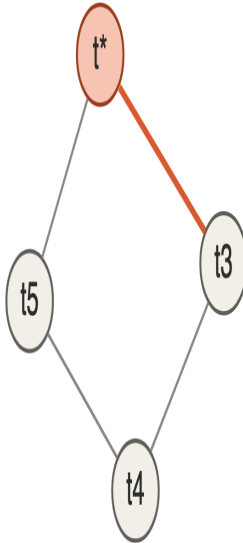
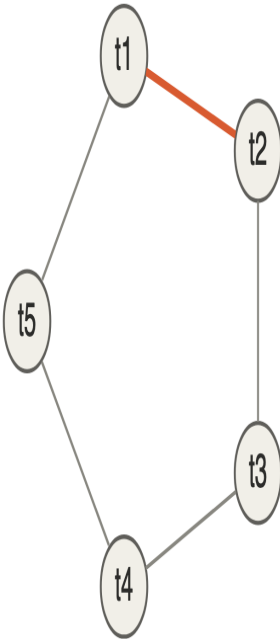


Figure 30: Contracting a cycle down to K_3 . The highlighted edge is the one contracted at each step.

4.4.5 Step 2: G has no $K_{1,3}$ minor, so $\Delta(G) \leq 2$

Suppose that $\Delta(G) \geq 3$. Then some vertex $v \in V(G)$ is adjacent to at least 3 distinct neighbors p, q, r . Delete all other vertices besides v, p, q and r . Delete any edges among p, q, r . We have 3 edges left, vp, vq and vr , which is $K_{1,3}$. Because we only deleted edges and vertices, $K_{1,3}$ would be a minor of G . Thus, by contraposition, if G has no $K_{1,3}$ minor then $\Delta(G) \leq 2$.

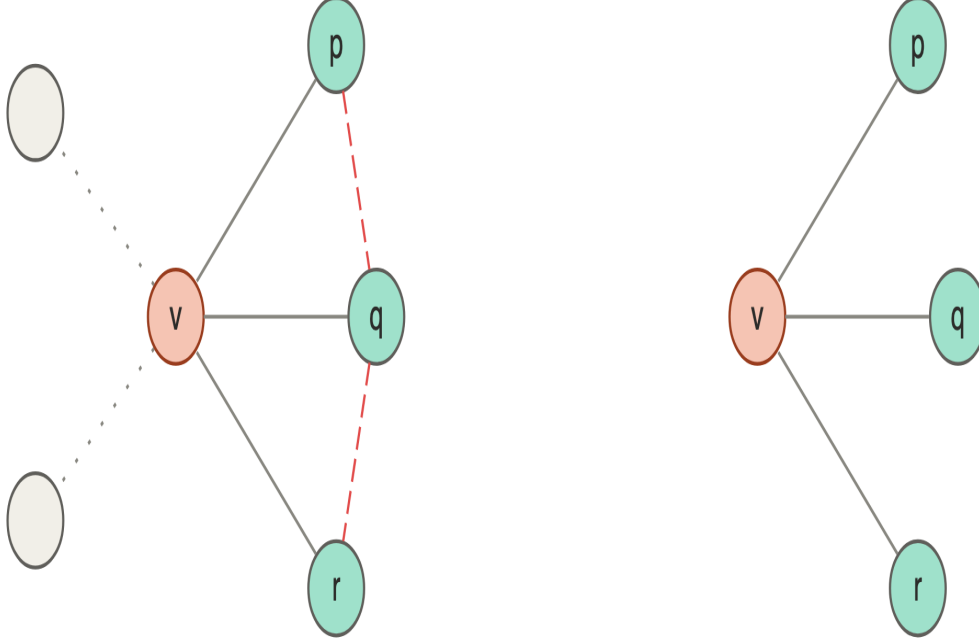


Figure 31: A vertex of degree 3 yields $K_{1,3}$ by deletions alone: no contraction needed.

4.4.6 Step 3: a forest with $\Delta \leq 2$ is a disjoint union of paths

Each connected component of G is a tree. Without loss of generality, pick one and label it T . Note that $\Delta(T) \leq 2$. If $|V(T)| = 1$, then $T = P_1$. Assume $|V(T)| \geq 2$. T is finite, so it must have some maximum path $P = v_0 v_1 \cdots v_L$.

Claim. $V(P) = V(T)$.

Suppose not. Since T is connected, there must be some vertex $z \notin V(P)$ and a path Q from z to a vertex v_i on P , where v_i is the first vertex where Q and P meet. There must be some u adjacent to v_i on Q , with $u \notin V(P)$. There are two cases.

Case 1: v_i is an endpoint of P . Say $v_i = v_L$; the argument for v_0 is symmetric. Then $v_0 v_1 \cdots v_L u$ is a path — no vertex repeats, since the v_j are distinct and $u \notin V(P)$ — with $|V(P)| + 1$ vertices. This path is longer than P , which is a contradiction to P being the longest path.

Case 2: v_i is a middle vertex of P . Thus v_i has at least two adjacent vertices v_{i-1} and v_{i+1} , where $v_{i-1}, v_{i+1} \in V(P)$. v_i is also adjacent to u . Thus $\deg(v_i) \geq 3$, which contradicts $\Delta(T) \leq 2$.

Both cases fail by way of contradiction, so $V(P) = V(T)$.

The edges. Sharing vertices is not yet sharing edges, so this needs one more line. P and T have the same number of edges, specifically because T is connected on $|V(P)|$ vertices, so it has exactly $|V(P)| - 1$ edges, and P already supplies that many. The only way T could have an extra edge is if that edge joined two vertices of P at distance ≥ 2 along P — a **chord** — but a chord $v_i v_j$ with $j \geq i + 2$ closes the cycle $v_i v_{i+1} \cdots v_j v_i$, and G is acyclic. So $E(T) = E(P)$ and $T = P$.

Therefore every tree in the forest is a single path, which means G is a union of disjoint paths.

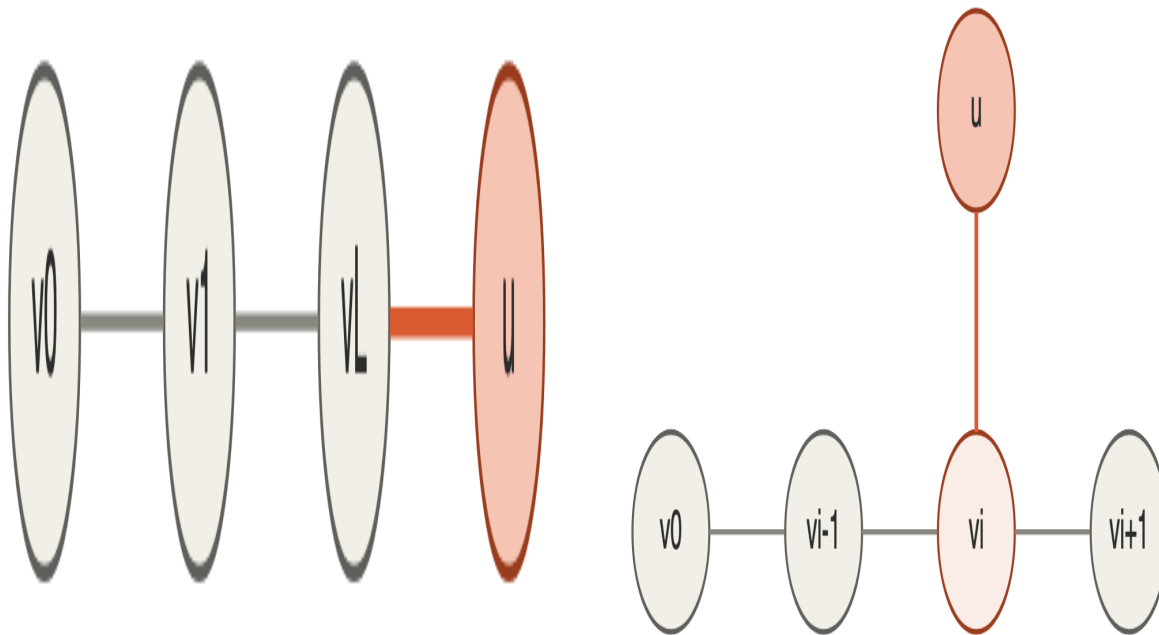


Figure 32: Case 1: u attaches at an endpoint, so P extends

Figure 33: Case 2: u attaches in the middle, so $\deg(v_i) \geq 3$

4.4.7 Step 4: Every path is conflict-free colorable with one color

Now we prove that all paths are conflict-free colorable using 1 color. A path on $3n$ vertices, $n \in \mathbb{Z}^+$, can be colored by coloring the middle vertex of every 3 vertices. Then every vertex has a conflict-free neighbor: each colored vertex covers exactly its own block of three, and the blocks tile the path with no gaps and no overlaps.

For a path on $3n - 1$ vertices, we can delete one endpoint from the $3n$ path, noting this vertex is uncolored, so we maintain our 1-CF-coloring. For $3n - 2$ we drop the remaining uncolored endpoint. Since

$$3n \equiv 0, \quad 3n - 1 \equiv 2, \quad 3n - 2 \equiv 1 \pmod{3},$$

and all integers are in $[0], [1], [2] \pmod{3}$, all paths on any number of vertices can be CF-colored with 1 color.

The block view, and what it tells you. With one color available the condition “some color occurs exactly once in $N[v]$ ” reduces to $|S \cap N[v]| = 1$ for every v , where S is the colored set. Equivalently, the closed neighborhoods $\{N[y] : y \in S\}$ **tile** the path. An interior colored vertex covers a block of 3; a colored vertex at an end covers a block of 2. So a valid coloring is exactly a decomposition of the path into consecutive blocks of size 3, with blocks of size 2 allowed **only at the two ends**:

$$2, \quad 3, \quad 4 = 2+2, \quad 5 = 3+2, \quad 6 = 3+3, \quad 7 = 2+3+2, \quad 8 = 3+3+2, \quad 9 = 3+3+3.$$

Counting the 2-blocks by residue explains how many valid colorings a path has. $N \equiv 0$ needs zero 2-blocks; $N \equiv 1$ needs two, one at each end and both forced; $N \equiv 2$ needs exactly one, which may sit at either end. So the coloring is **unique** when $N \equiv 0, 1 \pmod{3}$ and there are exactly **two** when $N \equiv 2$. Verified by exhaustive search for all $N \leq 24$.

Since the blocks tile, $|S| = \lceil N/3 \rceil$ for every valid coloring, which also computes the conflict-free domination number $\gamma_{CF}^1(P_N) = \lceil N/3 \rceil$.

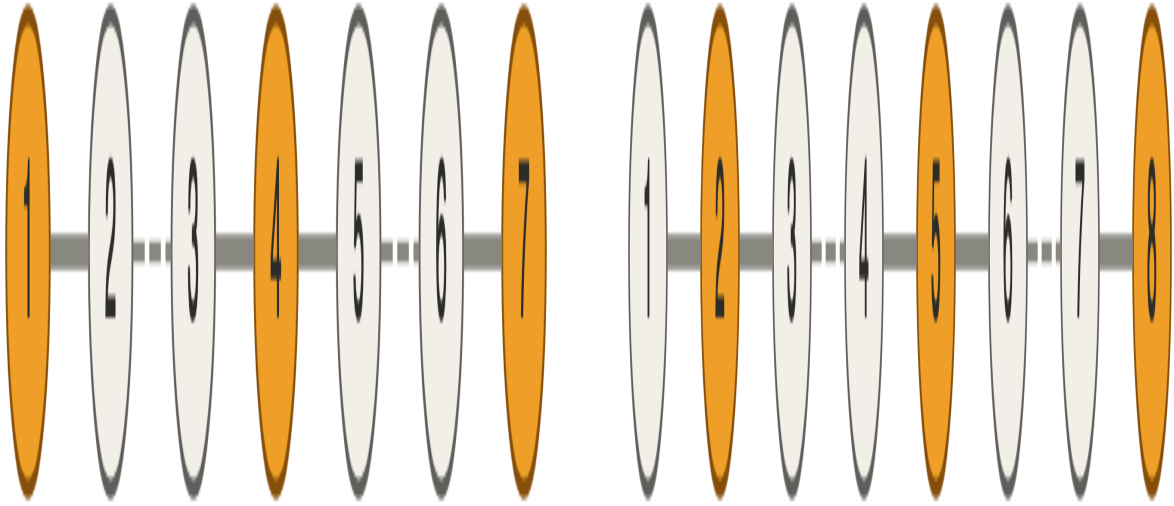


Figure 34: $P_7 = 2+3+2$. Solid edges lie inside a block; dashed edges cross between blocks. Figure 35: $P_8 = 3+3+2$. The other valid coloring is $2+3+3$.

This establishes $\mathcal{P}(1)$. ■

4.4.8 Distance-3 sets

Now let's construct a set $D \subseteq V(G)$, where D is the subset at pairwise distance at least 3 that uses color 1.

Disjoint neighborhoods. Any pair of vertices in D have disjoint neighborhoods, meaning no vertex in $N[v]$ is also in $N[y]$, so that

$$\text{dist}(v, y) \geq 3 \iff N[v] \cap N[y] = \emptyset.$$

Why: a vertex in $N[v] \cap N[y]$ is adjacent-or-equal to both, giving a path from v to y of length at most 2; conversely a path of length at most 2 hands you such a vertex.

$N[D]$ is the closed neighborhood of D : it includes D and the neighborhoods of every vertex $d \in D$.

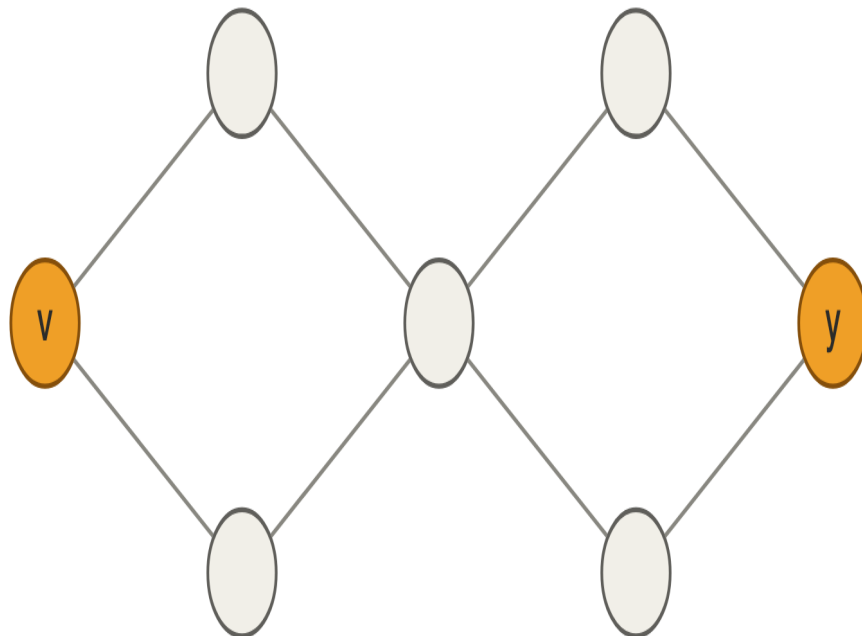


Figure 36: Two vertices at distance 3 have disjoint closed neighborhoods, so every vertex in $N[D]$ sees exactly one member of D .

Inclusion-maximality. D is inclusion-maximal, meaning that if any vertex were added, then D would not remain a distance-3 set. This means for any vertex $y \notin N[D]$, y is less than distance 3 from some vertex in D , because otherwise it would be in D ; and y cannot be distance 1 from D , because then y would be in D 's closed neighborhood. So

$$\text{dist}(y, D) = 2 \quad \text{for every } y \in W.$$

We call these vertices **unseen**, and write $W = V(G) \setminus N[D]$ for the set of them.

Cutset. $U \subseteq V(G)$ is a cutset of G if deleting U leaves G disconnected.

4.4.9 The algorithm

We can use an algorithm to make D inclusion-maximal and color the graph:

1. Set $i \leftarrow 1$ and $\chi \leftarrow \emptyset$. Remove all isolated paths from G and set them aside. **While G is not empty, do steps 2–5.**
2. Set $D \leftarrow \emptyset$. For each disjoint component of G , pick an arbitrary vertex and add it to D . Call the first such vertex the **seed**, s .
3. While there is a vertex y at distance ≥ 3 from all vertices in D , pick some y *at distance exactly 3 from some vertex already in D* and add it to D .
4. Assign $\chi(d) \leftarrow i$ for every $d \in D$, then set $i \leftarrow i + 1$.
5. Remove $N[D]$ from G . This ensures we only have vertices at distance 2 from D left. Then remove any newly isolated paths and set them aside.
6. **After the loop ends:** color all the isolated paths that were set aside, using the single color i , by the block decomposition of Section 4.4.3.

Every time the outer loop runs we add a new color. The first time, we color all the vertices in D and leave all $v \in N(D)$ uncolored, so they satisfy conflict-free coloring. We then delete $N[D]$, so D is empty again. We set any isolated paths to the side. Now D is empty again, and we repeat until G is empty.

We assign a new color each time to ensure that a vertex in $N(D_1)$ which is also in $N(D_2)$ remains CF-colored. If D_1 and D_2 both had color i , then i would not be unique.

We assign one more color at the end for the isolated paths, so

$$k = (\text{number of outer-loop runs}) + 1.$$

Why step 6 is outside the loop. We must use a different color i for each round. A covered vertex $v \in N[D_i] \setminus D_i$ can be adjacent to a path vertex stranded in the same round; that path vertex is not in $N[D_i]$, so it is a different vertex from v 's witness. If it received color i too, then v would see color i twice and lose its witness — and v has already been deleted, so nothing would catch it. Tested on random connected graphs, this variant produces an invalid coloring roughly 94% of the time. All stranded paths must share **one** color, assigned after the loop.

4.4.10 Inductive step: $\mathcal{P}(k-1) \Rightarrow \mathcal{P}(k)$

We've proved $\mathcal{P}(1)$ as a base case. Suppose $\mathcal{P}(k-1)$ was true. Now assume that G has no K_{k+2} and no K_{k+3}^{-3} minor. After one round we have W , which is an induced subgraph of G and therefore a minor of G , so W doesn't have K_{k+2} or K_{k+3}^{-3} as a minor.

That inherited fact is the **wrong size**, however. Substituting $k-1$ for k in $\mathcal{P}$'s hypothesis gives

$$K_{(k-1)+2} = K_{k+1}, \quad K_{(k-1)+3}^{-3} = K_{k+2}^{-3},$$

so what $\mathcal{P}(k-1)$ demands of W is that W avoid the *smaller* pair. Nothing about being a subgraph makes the largest clique shrink, so this has to be proved. That is the work of the next two claims.

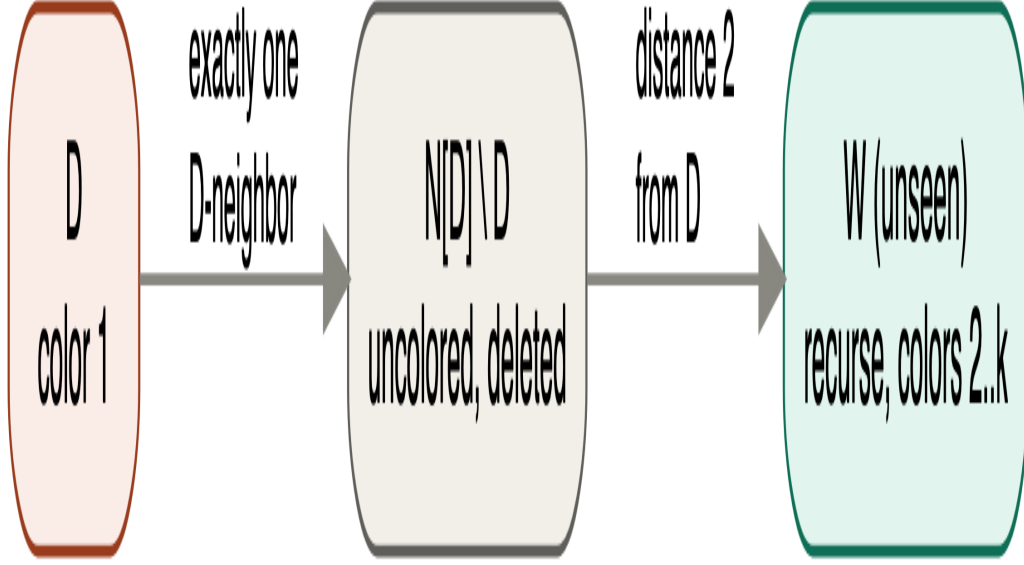


Figure 37: One round partitions the graph into three zones. Only D receives a color; the covered zone is deleted uncolored; the unseen zone is recursed on with fresh colors.

4.4.11 Claim A: no set of unseen vertices is a cutset of G

If G is disconnected, iterated elimination of distance-3 sets works on all components separately, so we can assume G to be connected.

Let U be a set of unseen vertices, i.e. $U \subseteq W$. For sake of contradiction, suppose U is a cutset. Then deleting U splits the graph. Pick a piece R that does not contain the seed s . Such an R must exist, because there are at least two pieces and the vertex s can only be in one of them.

Every vertex of R is at distance ≥ 3 from every marked vertex outside R . Any path from a vertex in R to a colored vertex in D must pass through U , by the definition of a cutset; then to a vertex in $N(D)$, the open neighborhood of D ; and then to a vertex in D . So the path from R to D is at least a distance of 3.

So some vertex of R has to be marked. At least one vertex of R is adjacent to U , and at least one vertex in $N(D)$ is adjacent to U ; and any vertex at least 3 away from every other marked vertex is exactly what step 3 of the algorithm adds to D . Since D is inclusion-maximal, no vertex can sit at distance ≥ 3 from all of D — so if R contained no marked vertex, every vertex of R would be addable, contradicting maximality.

The contradiction. Consider the iteration where the first vertex w of R is added to D . Moreover, w was not the seed s (the vertex arbitrarily chosen first by the algorithm), so by step 3, w is exactly 3 away from some marked vertex; call it d . Let a and b be the two interior vertices on the 3-edge path from w to d , so the path is $wab d$ with $a \in U$ and $b \in N(D)$. Then w is adjacent to a , and because w is marked, $a \in N[D]$, so $a \notin U$. This is a contradiction, so U cannot be a cutset. $\square$

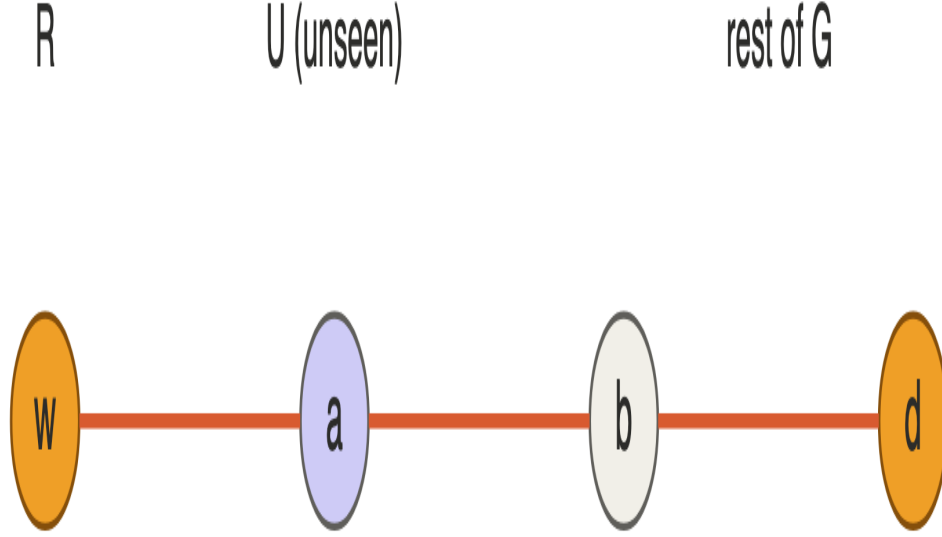


Figure 38: The 3-edge path $w a b d$ must cross U at its first step, which puts the unseen vertex a next to the marked vertex w .

This means that W , our set of unseen vertices, is also not a cutset since $W \supseteq U$. So $G \setminus W$ is connected.

Take $U = W \setminus \{y\}$ for each $y \in W$. U is still a set of unseen vertices, so it still isn't a cutset. Now take $G \setminus U = G \setminus (W \setminus \{y\})$, such that all unseen vertices but y are removed. A path from y to the seed s still exists, and it doesn't pass any other vertex in W . Because y is arbitrary, **every** $y \in W$ **has a neighbor outside of** W .

4.4.12 Claim B: W has no K_{k+1} minor and no K_{k+2}^{-3} minor

Suppose W has either forbidden minor. $W \neq V(G)$, since D is non-empty and $W \cap D = \emptyset$, so pick any vertex $f \in V(G) \setminus W$.

$G \setminus W$ is connected, so we can contract it into one vertex x . By Claim A, x is adjacent to all vertices of W . Adding this universal vertex to either forbidden graph enlarges it by 1:

$$K_{k+1} + x = K_{k+2}, \quad K_{k+2}^{-3} + x = K_{k+3}^{-3},$$

the second because x joins to everything, so the deleted triangle stays deleted. But we assumed these two minors do not occur in G , so by contradiction their smaller counterparts cannot occur in W . $\square$

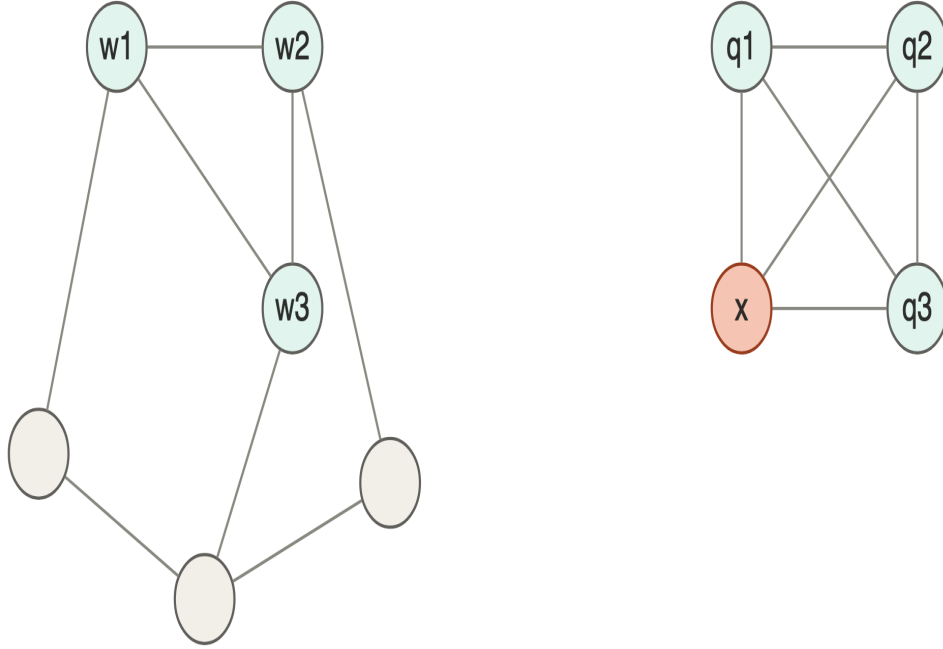


Figure 39: Contracting $G \setminus W$ to the single vertex x turns a K_{k+1} inside W into a K_{k+2} in G .

4.4.13 Assembling the induction

W now satisfies $\mathcal{P}(k-1)$, and D uses 1 color, so G is conflict-free k -colorable by induction, for all k , when it doesn't contain K_{k+2} and K_{k+3}^{-3} minors.

It remains to confirm that the combined coloring really is conflict-free, which the paper leaves implicit. There are two kinds of vertex.

Vertices in $N[D]$. Each has exactly one neighbor in D , so color 1 occurs exactly once in its closed neighborhood. Every later color is ≥ 2 , and adding occurrences of other colors can never destroy the uniqueness of color 1.

Vertices in W . Let $y \in W$ have conflict-free neighbor of color $c \geq 2$ from the recursive coloring of W . In G , the closed neighborhood $N[y]$ may be larger than it was inside W — but y is unseen, so y has **no** neighbor in D , and every extra neighbor lies in $N[D] \setminus D$, which is uncolored. So no new colors enter $N[y]$ at all, and c is still unique.

Every vertex has a conflict-free neighbor, so the coloring is valid and Theorem 3.5 holds. ■

Both excluded minors are necessary. G_{k+1} from Section 3 contains a K_{k+3}^{-3} minor but no K_{k+2} minor, and $\chi_{CF}(G_{k+1}) = k+1 > k$. So excluding K_{k+2} alone would make Theorem 3.5 false, and our own hierarchy is the counterexample. You can watch the induction stall: G_3 is series-parallel, hence K_4 -minor-free, so it passes the “no K_{k+2} ” half at $k=2$; but running one round leaves 33 unseen vertices containing a vertex of degree 3, hence a $K_{1,3}$ minor, so W fails $\mathcal{P}(1)$ and the recursion cannot bottom out.

The bound is an upper bound only. The algorithm's output depends on the arbitrary choices in steps 2 and 3, and can exceed $\chi_{CF}(G)$. On the graph with edges

13, 15, 17, 24, 25, 28, 34, 48, 58, 67, 78 the algorithm uses 3 colors while $\chi_{CF} = 2$. This is unavoidable: deciding $\chi_{CF}(G) \leq 1$ is NP-complete even for planar graphs, so no polynomial-time heuristic can be exact.

4.5 Corollary 3.6 — The Conflict-Free Hadwiger Theorem

Statement. All graphs that do not have K_{k+1} as a minor are conflict-free k -colorable. That is, $\chi_{CF}(G) \leq k$ whenever G has no K_{k+1} minor.

Proof. Assume G has no K_{k+1} minor. We check both hypotheses of Theorem 3.5.

No K_{k+2} minor. K_{k+1} is a subgraph of K_{k+2} , obtained by deleting one vertex, and every subgraph is a minor. Minor containment is transitive, so if K_{k+2} were a minor of G , then K_{k+1} would be too, contradicting the assumption.

No K_{k+3}^{-3} minor. Write K_{k+3}^{-3} on the vertex set $\{\alpha, \beta, \gamma\} \cup F$, where the triangle $\alpha\beta\gamma$ has been deleted and $|F| = k$. Then F is a clique, every vertex of F is adjacent to each of α, β, γ , and only α, β, γ are pairwise non-adjacent. Delete β and γ . What survives is $F \cup \{\alpha\}$: exactly $k + 1$ vertices, pairwise adjacent, so K_{k+1} . Hence K_{k+1} is a minor of K_{k+3}^{-3} , and the same transitivity argument applies.

Both hypotheses hold, so Theorem 3.5 gives a conflict-free k -coloring of G , and it is computable in polynomial time. ■

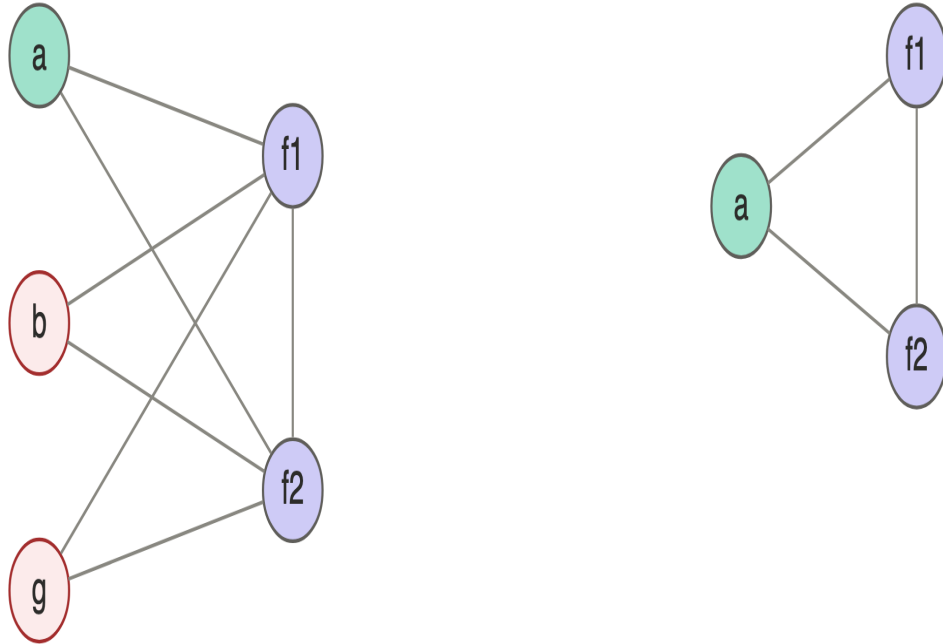


Figure 40: K_{k+1} is a minor of K_{k+3}^{-3} : delete two of the three independent vertices. Shown for $k = 2$, where K_5^{-3} reduces to K_3 .

Analogue to Hadwiger’s. The proper-coloring version of this statement — if G has no K_{k+1} minor then $\chi_P(G) \leq k$ — is **Hadwiger’s Conjecture**, open since 1943 and still unresolved for $k \geq 6$. The conflict-free version is a theorem with a page of proof. Given the inequality $\chi_{CF}(G) \leq \chi_P(G)$ from Section 3, this is a precise sense in which conflict-free coloring is the easier problem: the hard conjecture becomes provable.

Theorem 3.5 is strictly stronger than this corollary, since its hypothesis excludes a larger clique and therefore applies to more graphs. Corollary 4.4 of the paper draws the planar results from Theorem 3.5 the same way: planar graphs have no K_5 minor, and K_6^{-3} contains $K_{3,3}$, so planar graphs have no K_6^{-3} minor either — giving $\chi_{CF} \leq 3$ for planar graphs. Excluding K_4 and $K_5^{-3} \supseteq K_{2,3}$ likewise gives $\chi_{CF} \leq 2$ for outerplanar graphs.

5 Hardness

TODO: What’s being reduced from, and why that source problem. Reductions read as arbitrary until you see what structural feature is being exploited.

TODO: The gadget. Draw it. Then state what it enforces and verify both directions of the correctness argument separately — most write-ups compress these into one paragraph and lose the reader.

6 Outerplanar graphs

TODO: Why outerplanar is the tractable case. What does the structure buy you that general graphs don’t?

TODO: The dynamic programming pipeline. What’s the state, what’s the recurrence, what’s the order of evaluation, what’s the runtime. A worked example on a small outerplanar graph would carry this section.

7 Loose ends

TODO: Places where you’re unsure, steps you couldn’t fully reconstruct, open questions the paper raises. Keeping this section visible is a feature — it tells readers where the hard parts are and it’s honest about the limits of the reconstruction.

8 References